

MIZN

A modular spectral solver for the K=3
noiseless CRRA rational-expectations equilibrium

Economic model, mathematical formulation,
Chebyshev discretization, and numerical algorithm

Companion volume to MIZN repository

Contents

List of Figures

Preface

This book is the definitive technical companion to the MIZN codebase: an end-to-end treatment of the noiseless $K=3$ CRRA rational-expectations equilibrium (REE) problem, from economic foundations through to a working numerical algorithm based on symmetric Chebyshev spectral methods.

It is structured in seven parts:

Part I — Economic setup. The model, its primitives (agents, signals, asset), the information structure, individual demands, and the market-clearing problem. The 5-step Hellwig cycle as the natural fixed-point loop.

Part II — Mathematical formulation. The exact equation system. The price as a function $P(u_1, u_2, u_3)$. Boundary conditions and asymptotic limits. Symmetries (S_3 and Z_2). The relationship to known closed-form cases (CARA Hellwig 1980, fully revealing limit).

Part III — Chebyshev discretization. Coordinate transformations, the tensor Chebyshev basis, boundary conditions in Chebyshev, $S_3 \times Z_2$ symmetric basis, per-agent evaluation via permutation, companion-matrix contour root-find, Gauss-Legendre quadrature.

Part IV — Numerical algorithm. The full pipeline from P to $\Phi(P)$, slice-evidence computation, Bayesian update, CRRA clearing, dense Newton iteration, γ -continuation strategies.

Part V — Implementation in MIZN. The module structure, build order, testing strategy, the per-run execution-log system with auto-generated LaTeX PDF reports.

Part VI — Validation. CARA Hellwig closed-form gold test, symmetry checks, spectral-decay diagnostics, γ -sweep reproduction of prior machine-precision results.

Part VII — Outlook. Past the γ -fold via pseudo-arclength continuation; higher K extensions; the relationship to other REE models.

The text assumes graduate-level familiarity with Bayesian asset pricing and basic numerical linear algebra (Newton’s method, Chebyshev polynomials), but attempts to develop every nontrivial step in full so that a careful reader can implement MIZN from scratch using this document as the specification.

Part I

Economic Setup

Chapter 1

The Model: Primitives and Information Structure

1.1 What problem are we solving?

A canonical question in financial economics is whether — and how completely — equilibrium asset prices aggregate the dispersed private information of many traders. In a *fully revealing* equilibrium (FR), the price is a sufficient statistic for the joint information of all agents: an outsider observing only P knows as much about the asset value as the union of all agents' signals. In a *partially revealing* equilibrium (PR), the price reveals only some — typically the projection along one linear combination of signals — leaving residual uncertainty even to a fully-Bayesian outside observer.

Whether REE is FR or PR depends on the joint structure of preferences, signal distributions, and the asset payoff. Two seminal results frame the landscape:

- **Hellwig (1980).** In a Gaussian-signal Gaussian-supply-noise linear REE with CARA agents, the equilibrium is partially revealing because supply noise injects uncertainty into the price even when agents know the signal distribution. Hellwig derived the linear coefficients in closed form; this serves as our gold test (Chapter ??).
- **Grossman & Stiglitz (1980); paradox of informationally efficient markets.** If REE is FR, no one has an incentive to gather information. Equilibrium markets cannot be too informationally efficient.

The MIZN project tackles a less-studied variant: **noiseless K=3 CRRA REE with a binary asset value** $v \in \{0, 1\}$. Removing supply noise removes Hellwig's smoking gun for PR; replacing CARA with CRRA + a binary asset introduces a Jensen-gap mechanism (concavity of demand in beliefs) that *may* generate PR for sufficiently low risk aversion γ . Whether this PR equilibrium genuinely exists — and what its properties are — is the substantive question, and answering it numerically requires a robust fixed-point solver. That is MIZN.

1.2 Primitives

1.2.1 The asset

A single risky asset has uncertain payoff

$$v \in \{0, 1\}, \quad \Pr(v = 0) = \Pr(v = 1) = \frac{1}{2}. \quad (1.1)$$

The binary structure simplifies Bayesian arithmetic: each posterior is a single number $\mu \in [0, 1]$ representing $\Pr(v = 1 \mid \text{info})$.

1.2.2 The agents and their signals

There are $K = 3$ symmetric agents indexed by $k \in \{1, 2, 3\}$. Each agent k observes a private noisy signal:

$u_k = v + \varepsilon_k$, $\varepsilon_k \sim \mathcal{N}(0, \tau^{-1})$, (1.2) with the noises ε_k mutually independent given v . The *signal precision* τ is a model parameter; in our worked example we take $\tau = 2$.

Conditional on v , the signal

$$u_k$$

is Gaussian with mean v and variance $1/\tau$. Writing the conditional density as f_v :

$$f_v(u) = \sqrt{\frac{\tau}{2\pi}} \exp\left(-\frac{\tau}{2}(u - m_v)^2\right), \quad m_0 = -\frac{1}{2}, \quad m_1 = +\frac{1}{2}. \quad (1.3)$$

(Note: the convention $m_0 = -1/2$, $m_1 = +1/2$ rather than $m_0 = 0$, $m_1 = 1$ is a centered re-parametrization; the relevant economics — the difference $m_1 - m_0 = 1$ and the precision τ — is unchanged.)

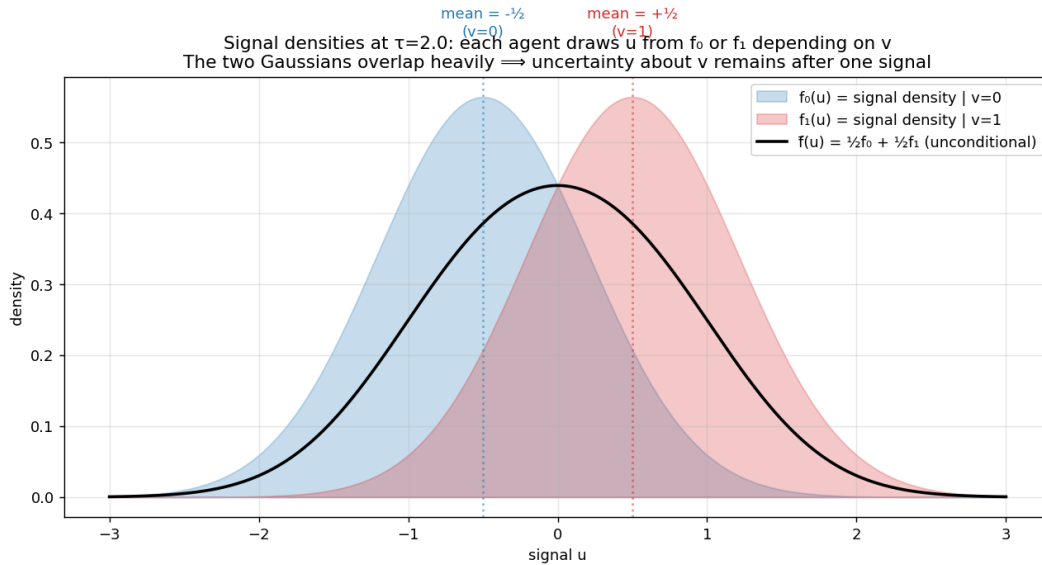


Figure 1.1: The two conditional signal densities f_0 (blue) and f_1 (red), at $\tau = 2$. The unconditional density $\bar{f} = \frac{1}{2}f_0 + \frac{1}{2}f_1$ (black) is a Gaussian mixture peaked between the two component means. The substantial overlap of f_0 and f_1 is the reason a single signal does not fully reveal v .

1.2.3 Preferences (CRRA)

Each agent has CRRA preferences with constant risk-aversion parameter $\gamma > 0$. For a binary asset paying $\{0, 1\}$, the demand derived from maximizing expected log-of-end-of-period-wealth (or any CRRA utility — the binary-payoff case collapses preferences to a clean parametric form) is

$$x_k(u_k, P; \gamma) = u_k - P \frac{1}{\gamma} u_k(1-u_k), \quad (1.4)$$

where u_k is agent k 's posterior belief about $v=1$, P is the price, and γ is risk aversion. The denominator $u_k(1-u_k)$ is the binomial variance of the asset payoff f under agent k 's belief; the form is the standard mean-variance demand specialized to a binary payoff f .

Remark 1.1. In the literature this CRRA-on-binary form is sometimes called "log-utility CRRA" or just "CRRA demand" without further qualification. The key features are: (a) demand is linear in the wedge $\mu - P$; (b) demand scales inversely with risk aversion γ ; (c) demand vanishes when $\mu = P$ (the agent is indifferent). When $\gamma \rightarrow \infty$ (extreme risk aversion) the demand goes to zero — agents refuse to trade unless prices and beliefs are infinitely misaligned. When $\gamma \rightarrow 0$, the demand becomes infinite for any $\mu \neq P$ — the agents become arbitrageurs.

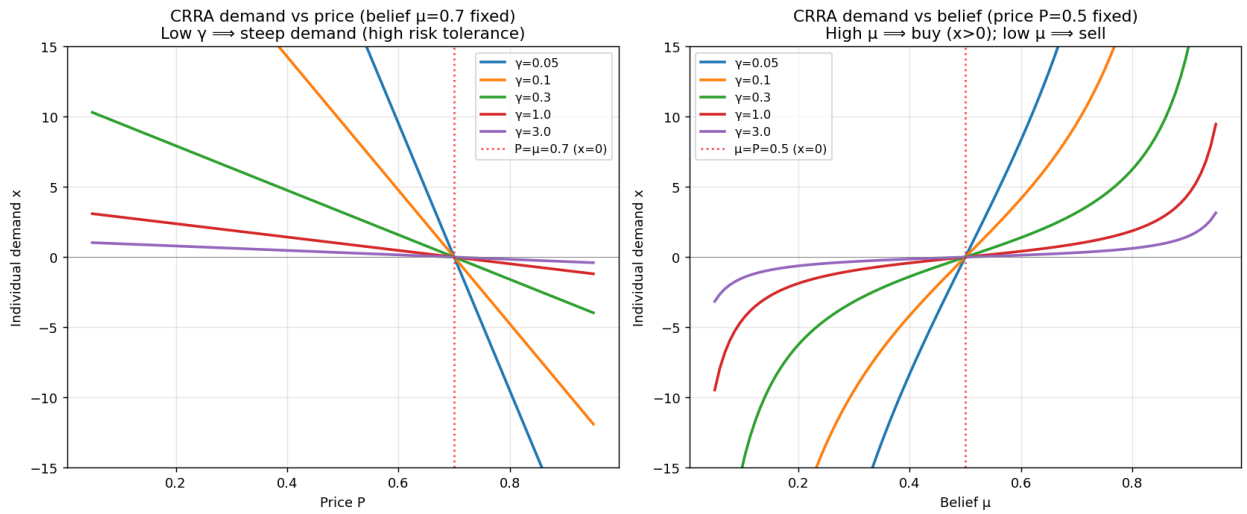


Figure 1.2: CRRA demand $x = (\mu - P)/(\gamma\mu(1 - \mu))$. Left: as a function of price P for fixed belief $\mu = 0.7$, at several values of γ . The demand crosses zero at $P = \mu$. Right: same as a function of belief μ for fixed price $P = 0.5$. Lower γ gives steeper demand.

1.2.4 Market structure (continuum, then $K=3$ stylized)

We adopt the standard "continuum of traders" simplification: there is a continuum of traders of each type $k = 1, 2, 3$, and within each type all traders see the same signal

$$u_k$$

. By the law of large numbers applied across the continuum, aggregating demands gives a finite per-type quantity, and the equilibrium condition (zero aggregate excess demand) becomes a clean three-equation system. Equivalently and more economically, think of the three agents as representative agents of three large equally-sized groups whose private signals happen to be drawn iid.

1.3 Information timeline

Within one trading period the order of events is:



Figure 1.3: The K=3 CRRA REE timeline within one trading period. Nature draws v ; each agent receives one noisy signal

$$u_k$$

; agents simultaneously form posteriors u_k (combining their own signal with the conjectured price function); they submit their demands $x_k(u_k, P)$. The market clearing condition $\sum_k x_k = 0$ determines the clearing price P . The fixed-point loop iterates on the conjecture.

1.4 The rational-expectations condition

The defining feature of REE is the consistency requirement: agents' *conjecture* of the price function $P(\cdot)$ must equal the price function that emerges from the market clearing when each agent uses this conjecture in their Bayesian update. Formally, with the conjectured price function denoted $P^{\text{conj}}(u_1, u_2, u_3)$ and the price function that clears the market $P^{\text{cleared}}(u_1, u_2, u_3)$, REE requires

$$P^{\text{conj}} \equiv P^{\text{cleared}}. \quad (1.5)$$

This is a fixed-point condition. The solution P^* is called the *rational-expectations equilibrium price function*, and is what MIZN is designed to compute.

1.5 The 5-step Hellwig cycle

(??) is a single fixed-point statement; in practice, computing P^* proceeds via an iterative cycle of five steps:

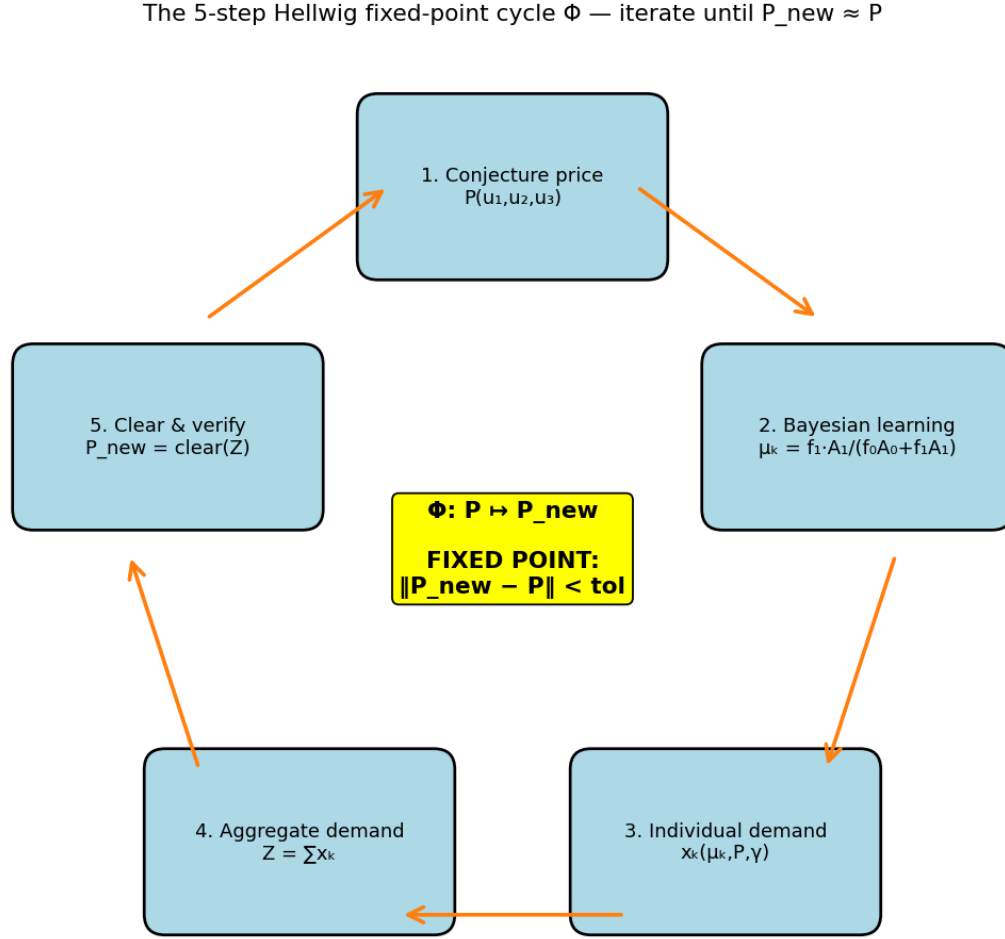


Figure 1.4: The 5-step Hellwig fixed-point cycle. Each iteration: (1) start from a conjectured price function P ; (2) compute the implied Bayesian posteriors u_k for each agent; (3) compute individual CRRAd demands x_k ; (4) aggregate demands; (5) clear the market to obtain P_{new} . Check $\|P_{\text{new}} - P\| < \text{tol}$; if not, update P and repeat. This is the operator $\Phi: P \mapsto P_{\text{new}}$ whose fixed point is the REE.

We will return to this cycle as the architectural backbone of MIZN (Chapter ??). For now, the cycle defines the operator Φ in the abstract; the next chapter develops the Bayesian-learning step in detail, which is by far the most numerically demanding.

Chapter 2

The Learning Step: Bayesian Update Given a Price Function

2.1 The agent's inference problem

Suppose agent k has just received their signal

$$u_k$$

and is about to submit their demand. They are about to do so simultaneously with the other agents; from their perspective, P is a function of all three signals (u_1, u_2, u_3) (the conjectured price function) but they only observe

$$u_k$$

. Their information set is $\mathcal{I}_k = \{u_k, P(\cdot)\}$ — their own signal, and the conjectured form of the price function.

The agent reasons as follows: "the realized price will be $P_{\text{realized}} = P(u_1, u_2, u_3)$ for the realized (u_1, u_2, u_3) . But I observe P_{realized} as part of the market, so I can use it to learn about the other agents' signals — and hence about v ." This is the rational-expectations step: the price is informative because it depends on the unobserved u_j for $j \neq k$.

Remark 2.1 (The chicken-and-egg). The price function $P(\cdot)$ that agent k uses for inference is the *conjectured* price function, which (in equilibrium) is the same one that emerges from market clearing given everyone's inference. This circularity is exactly what makes REE a fixed-point problem; the 5-step cycle resolves it by iterating until conjecture and cleared price coincide.

2.2 Bayesian posterior in terms of evidence ratios

Agent k 's posterior on $v = 1$, conditional on their information set $\mathcal{I}_k = \{u_k, P=p\}$ (*treating the realized price as a draw*)
 $= \Pr(v = 1 \mid u_k, P = p) = \Pr(u_k \mid v = 1) \cdot \Pr(P = p \mid v = 1) / \Pr(u_k \mid v = 0) \Pr(P = p \mid v = 0) + \Pr(u_k \mid v = 1) \Pr(P = p \mid v = 1)$ (2.1) using the symmetric prior $\Pr(v = 0) = \Pr(v = 1) = 1/2$ (which cancels).

The first factor in each numerator/denominator is the standard Gaussian density $f_v(u_k)$. *The second factor* — $\Pr(P = p \mid v)$ — is the density of the price conditional on v , integrating over the unobserved signals of agents other than k .

2.3 The co-area form of the conditional price density

For a fixed conjectured price function $P(u_1, u_2, u_3)$, the conditional density of P given v is, by change of variables / co-area formula,

$$\Pr(P = p \mid v) = \int_{\{P(u_{-k})=p\}} f_v(u_a) f_v(u_b) \frac{d\sigma}{|\nabla_{(u_a, u_b)} P|}, \quad (2.2)$$

where $u_{-k} = (u_a, u_b)$ are the two signals not seen by agent k , the integration is over the level set (contour) where P equals the target price p , $d\sigma$ is the 1-D arc-length measure along the contour, and $|\nabla P|$ is the gradient magnitude of P in the (u_a, u_b) plane.

This is the central object of the learning step. Concretely:

- For agent 1, the slice is over (u_2, u_3) at fixed u_1 .
- For agent 2, over (u_1, u_3) at fixed u_2 .
- For agent 3, over (u_1, u_2) at fixed u_3 .

We denote the value of this integral, normalized by symmetry, as $A_v(p; u_k)$ — the evidence for v at price p given u_k . (2.3)

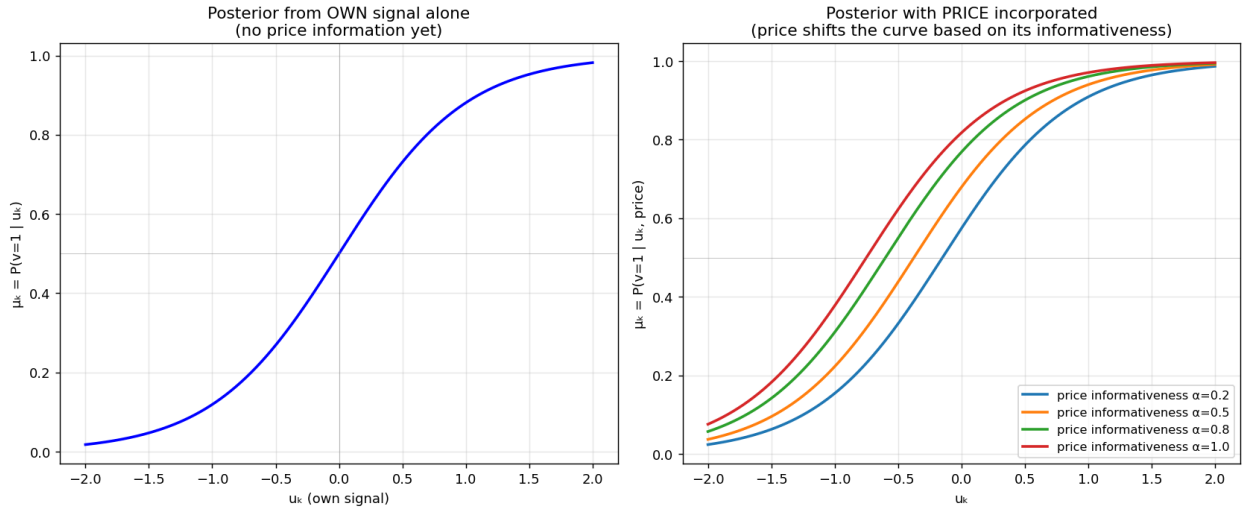


Figure 2.1: Posterior beliefs u_k as function of the own signal u_k . Left : posterior from the signal alone (no price information). Right : posterior incorporating price information at various levels. Higher $\alpha \Rightarrow$ more informative price $\Rightarrow$ posterior shifts more.

2.4 Why this integral is hard

The integral (??) is at the heart of the numerical difficulty that animates this entire book:

- The integrand contains $1/|\nabla P|$, which can blow up at *Morse-critical* points where the gradient vanishes (saddles/extrema of the surface $P(u_a, u_b)$).

- The contour $\{P = p\}$ is a level set in 2-D — its topology can *change* as p varies: contour components are born, merged, or die at Morse-critical price values. The integral $A_v(p)$ is mathematically a continuous function of p (this is a consequence of the co-area formula and Federer’s theorem), but *a numerical implementation that finds contour roots component-by-component* has discrete jumps in the root count as p crosses critical values.
- The contour must be located at machine precision for derivatives $|\nabla P|$ to be accurate; small errors in root location induce large errors in A_v if $|\nabla P|$ is small near that root.

These three issues drove the entire previous research arc (the ”prior session” referenced throughout this book), motivating the spectral discretization developed in Part ??.

Chapter 3

Demand, Aggregation, and Market Clearing

3.1 Step 3: individual CRRA demand

Given each agent's posterior u_k (computed in Chapter ??), their submitted demand is (??): $x_k = u_k - P_{\bar{\gamma}} u_k (1 - u_k)$. (3.1)

Two observations to keep in mind:

Sign of demand.

$$x_k$$

0 if $f(u_k) \leq P$; the agent buys when their belief is more optimistic than the price.

The "Jensen gap" mechanism. Because $u_k(1-u_k)$ is concave in u_k , an agent who is uncertain (u_k near $1/2$) has high "effective variance" and demands less aggressively per unit of misalignment. This concavity is what differentiates CRRA from CARA in our problem: it introduces a curvature into the aggregate demand that, in principle, can support PR equilibria where CARA (linear-in-beliefs aggregation) would not.

3.2 Step 4: aggregation

For the $K=3$ (one-representative-agent-per-type) variant, aggregation is simply summing:

$$Z = \sum_{k=1}^3$$

$$x_k = \sum_{k=1}^3 u_k - P_{\bar{\gamma}} u_k (1 - u_k). \quad (3.2)$$

For the continuum-of-traders variant, the same expression emerges as the average across types (with the per-type continuum collapsing to the representative-agent demand via LLN).

3.3 Step 5: market clearing

Market clearing is the requirement that aggregate (excess) demand equals zero:

$$Z(\mu_1, \mu_2, \mu_3; P) = 0. \quad (3.3)$$

For each fixed triple (μ_1, μ_2, μ_3) , equation (??) defines P implicitly as the price that clears the market. Since the demand function is monotonically decreasing in P (each term $\partial x_k / \partial P = -1/(\gamma u_k(1-u_k)) < 0$), the solution P^* is unique in $(0, 1)$.

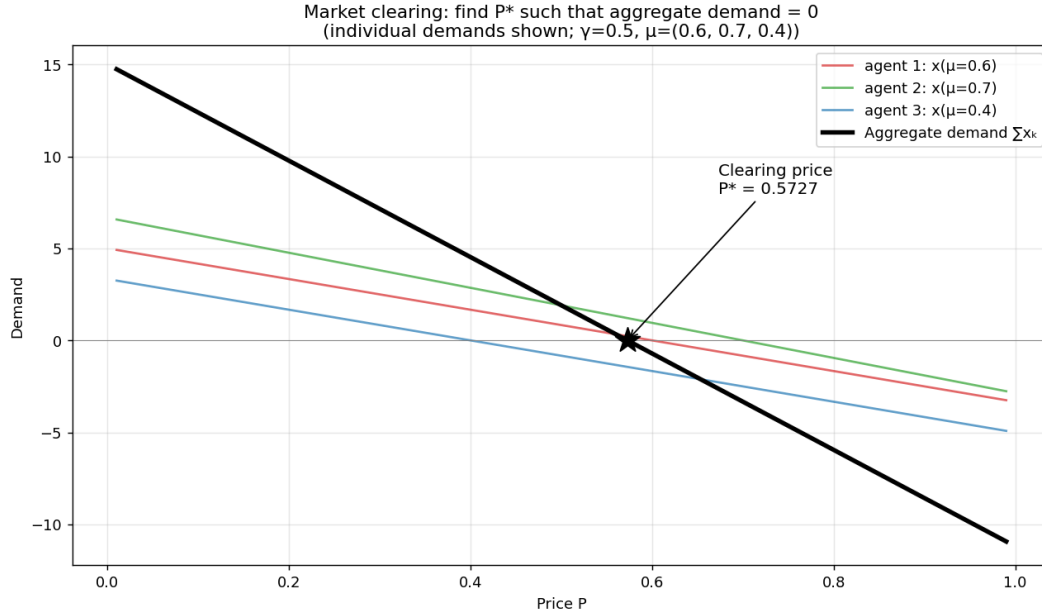


Figure 3.1: Market clearing: individual demands

$$x_k$$

(P ; u_k) as functions of price (colored), and their sum $Z = \sum_k x_k$ (black). The clearing price P^* is where the aggregate demand crosses zero.

3.4 Implementation: a bisection on log-odds

Numerically, market clearing is implemented as a bisection on the log-odds of P (the choice of variable matters for numerical robustness when u_k are near 0 or 1):

[title=**CRRA market-clearing bisection (matches crra_clear_nb in MIZN)**, colback=gray!5, colframe=black] Given (μ_0, μ_1, μ_2) and γ :

1. Compute log-odds $\ell_k = \log(u_k / (1-u_k))$ for each agent. Bisection $m \in [\varepsilon, 1 - \varepsilon]$:
[label=()]
2. (a) compute $\ell_P = \log(m/(1-m))$,
 (b) compute $R_k = \exp((\ell_k - \ell_P)/\gamma)$,
 (c) compute aggregate excess $E = \sum_k (R_k - 1)/((1-m) + R_k m)$.

(d) if $E > 0$: lower half; else: upper half. 200 iters $\rightarrow$ machine precision.

3. Return $P = (a + b)/2$.

This is robust, branch-free except for the bisection direction, and extends to arbitrarily many agents.

Chapter 4

The Two-Basin Phenomenon and the γ -Fold

4.1 Equilibrium is not unique: the PR and near-FR basins

A finding from the prior session that fundamentally shapes the design of MIZN: at $\gamma = 0.1, \tau = 2$ the operator Φ has *at least two fixed points*, in distinct basins of attraction:

- A **deep PR fixed point**: $\text{slope}_T \approx 0.36$, deficit ≈ 0.17 , distance to FR ≈ 0.26 . Reachable via Newton from a kernel-smoothed warm-start.
- A **near-FR fixed point**: $\text{slope}_T \approx 0.85$ to 0.95 , deficit ≈ 0.03 to 0.05 . Reachable via Picard from a no-learning initial condition.

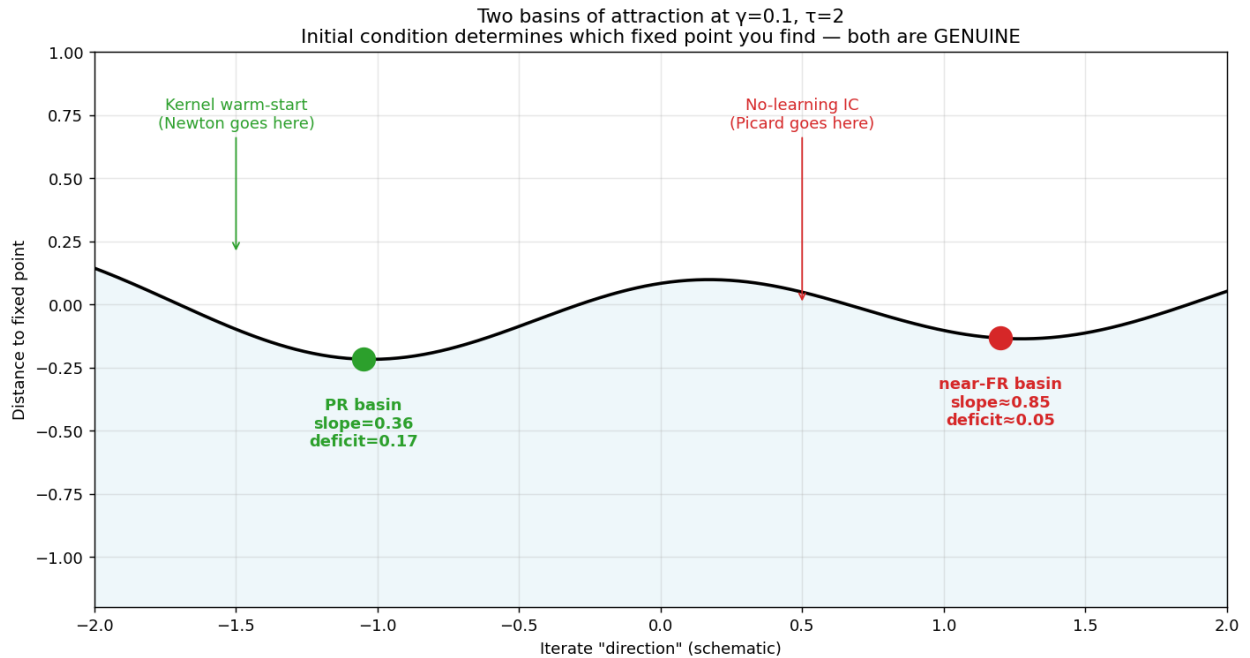


Figure 4.1: Two basins of attraction at $\gamma = 0.1, \tau = 2$ (schematic). **Both fixed points are genuine.** Which one a solver finds depends entirely on its starting point and method.

This is the single most important practical lesson from the prior work: **a solver without a kernel warm-start lands in the wrong basin.** We will return to this in Chapter ?? (algorithm design) and again in Chapter ?? (validation).

4.2 The γ -fold

Continuing the deep-PR fixed point in γ — a “ γ -sweep” — works smoothly across $\gamma \in [10^{-3}, 0.26]$, all 16 verified points nailed to $\|F\|_\infty \leq 10^{-11}$ at $G=9$. At $\gamma \approx 0.27$ the deep PR branch destabilizes: adaptive-step continuation cannot push past this γ . From the CARA side ($\gamma \rightarrow \infty$, FR ansatz), Newton-Krylov descent finds a different PR branch (slope ≈ 0.66) that exists down to about $\gamma = 3$ but not below.

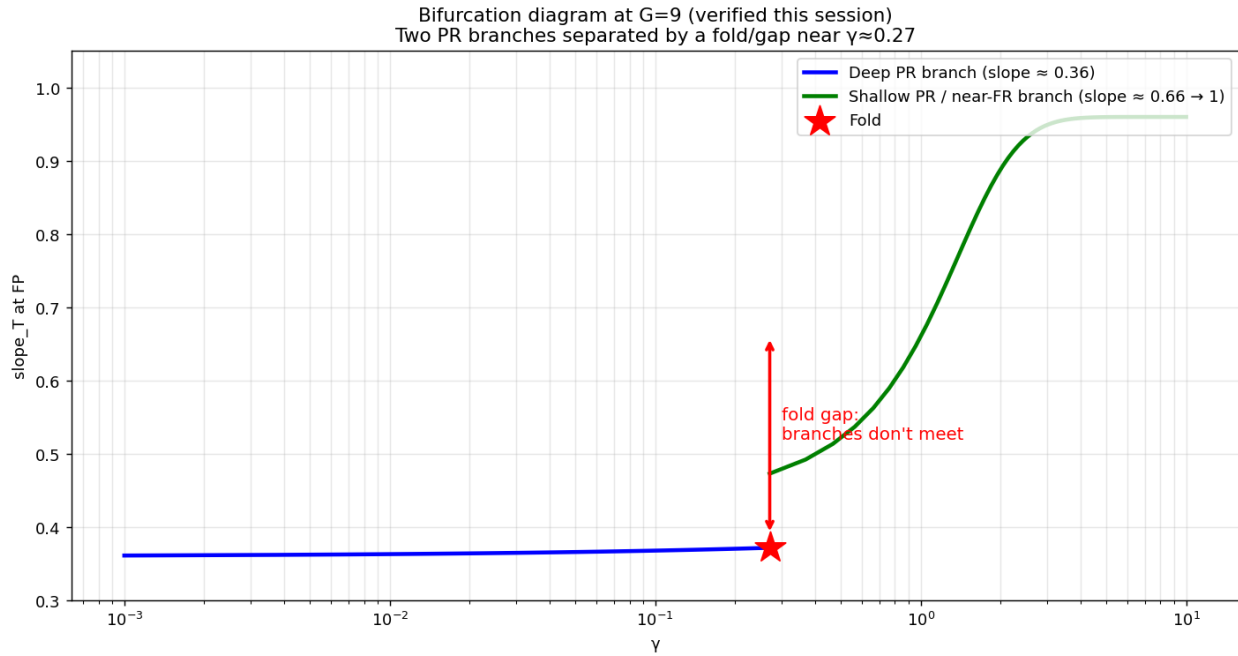


Figure 4.2: Bifurcation diagram at $G=9$ (verified this session). Two PR branches separated by a fold/gap near $\gamma \approx 0.27$. The deep PR branch (blue) is reachable from low- γ continuation. The shallow PR branch (green) is reachable from CARA-limit warm-start. Whether the gap is genuine (true bifurcation) or numerical (artifact of finite G) is the question we want MIZN to resolve.

We will discuss strategies for traversing the fold in Chapter ??.

4.3 Limits as $\gamma \rightarrow 0$ and $\gamma \rightarrow \infty$

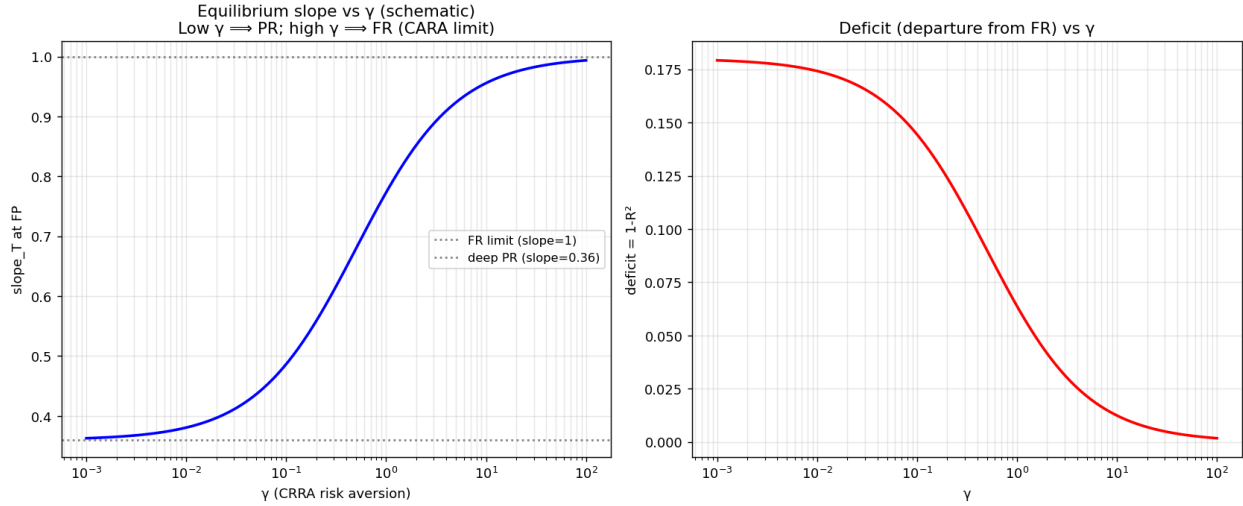


Figure 4.3: Schematic behavior of equilibrium slope and deficit as functions of γ . As $\gamma \rightarrow \infty$ (CARA limit), Hellwig’s argument predicts FR (slope = 1, deficit = 0). As $\gamma \rightarrow 0$, the equilibrium becomes maximally PR. The deep-PR branch verified this session has slope $\rightarrow 0.3599$ as $\gamma \rightarrow 0$.

$\gamma \rightarrow \infty$ (**CARA limit**). CRRA demand $x \propto (\mu - P)/(\gamma\mu(1 - \mu)) \rightarrow 0$ for any finite wedge. To support trade, the implied price must equal the harmonic-precision-weighted average of beliefs — which under Gaussian-signal Hellwig is fully revealing (slope = 1). **This is the CARA = FR result.**

$\gamma \rightarrow 0$ (**risk-neutral limit**). Demand becomes infinite for any finite wedge, so agents are arbitrageurs. Equilibrium requires $u_k = P$ for all k , which is unattainable unless the price reveals sufficient information — i.e. FR again. But this argument fails at finite γ because of the Jensen-gap mechanism that supports PR.

The verified asymptote slope $\rightarrow 0.3599$ as $\gamma \rightarrow 0$ in our sweep is a striking empirical result with no closed-form derivation we know of.

Part II

Mathematical Formulation

Chapter 5

The System of Equations

5.1 The unknown: a function on the cube

The unknown of the problem is the price function

$$P : \mathbb{R}^3 \rightarrow [0, 1], \quad (u_1, u_2, u_3) \mapsto P(u_1, u_2, u_3). \quad (5.1)$$

We discretize this on a cube. Three coordinate choices were investigated in the prior session (Chapter ??), but for the spectral approach we settle on the σ - δ atanh-stretched ξ -cube $(\xi_{u_1}, \xi_\Sigma, \xi_\delta) \in [-1, 1]^3$.

5.2 The full operator equation

The 5-step Hellwig cycle from Figure ?? can be written as a single fixed-point equation. Defining the operator $\Phi : \{P : \text{cube} \rightarrow [0, 1]\} \rightarrow \{P\}$ pointwise as

$$\Phi[P](u_1, u_2, u_3) = P_{\text{clear}}(\mu_1(u_1, P), \mu_2(u_2, P), \mu_3(u_3, P); \gamma) \quad (5.2)$$

the REE problem is

$$\boxed{\Phi[P^*] = P^*}. \quad (5.3)$$

Here:

- $u_k(u_k, P)$ is the posterior(??), which depends on P through the slice-evidence integral(??). P_{clear} is the market-clearing function defined implicitly by (??).
- All quantities depend on (u_1, u_2, u_3) pointwise through the u_k , which depend on the full functional P . This is a **nonlinear, integral fixed-point equation for a function on a 3-D domain**. There is no closed form except in special limits (CARA $\gamma \rightarrow \infty$, no information $\tau \rightarrow 0$, etc.).

5.3 Boundary conditions

The cube $\mathbb{R}^3$ extends to infinity; the natural "boundaries" are the limits

$$u_k$$

$\rightarrow \pm\infty$:

$u_k \rightarrow +\infty$ (**any single k**): agent k 's signal is infinitely positive, certifying $v = 1$ to agent k . The other agents' information remains finite. The price approaches the $K=2$ sub-problem solution conditional on $u_k = 1$.

$u_k \rightarrow -\infty$: symmetric, $u_k = 0, K = 2$ sub-problem conditional.

All three $u_k \rightarrow \pm\infty$ in the same direction: all agents certain of the same value; $P = 0$ or 1 exactly.

Mixed limits (e.g., $u_1 \rightarrow +\infty, u_2 \rightarrow -\infty, u_3$ finite): two agents have opposite-direction infinite signals; their beliefs cancel; P is determined by the remaining finite-signal agent and the residual prior. Non-trivial; depends on γ .

In the bounded ξ -cube these become "Dirichlet-like" conditions on the faces of the cube. They are not all simple constants; some are functions on the cube's faces.

5.4 Symmetries

Two symmetries of the operator Φ :

S_3 (permutation of agents). For any permutation $\sigma \in S_3$ of $\{1, 2, 3\}$,

$$\Phi[P_\sigma] = (\Phi[P])_\sigma, \quad \text{where} \quad P_\sigma(u_1, u_2, u_3) = P(u_{\sigma(1)}, u_{\sigma(2)}, u_{\sigma(3)}). \quad (5.4)$$

Consequence: the fixed point can be chosen S_3 -symmetric.

Z_2 (parity). The binary asset value's $v \leftrightarrow 1 - v$ symmetry implies

$$P(-u_1, -u_2, -u_3) = 1 - P(u_1, u_2, u_3). \quad (5.5)$$

Consequence: in the spectral expansion, half the coefficients are zero (those of even total order; see Sec. ??).

Chapter 6

The CARA Hellwig Closed Form

6.1 Why we need a closed form for testing

The CARA-Gaussian-Gaussian Hellwig model has a closed-form linear REE; this is the gold standard against which every component of MIZN must be tested. The MIZN architecture starts with the CARA case as the working warm-up, because every step has an analytic ground truth.

6.2 The CARA-Gaussian-Gaussian model

Primitives (slightly different from the K=3 binary-asset CRRA model):

- Payoff $\theta \sim \mathcal{N}(\bar{\theta}, 1/\tau_\theta)$ Gaussian.
- Each agent: signal $s_i = \theta + \varepsilon_i$, $\varepsilon \sim \mathcal{N}(0, 1/\tau_\varepsilon)$.
- Supply noise: $u \sim \mathcal{N}(\bar{u}, 1/\tau_u)$.
- CARA preferences with risk aversion ρ .
- Continuum of traders.

6.3 The closed-form linear REE

Conjecture $P = a + b\theta - cu$. Then the price-implied signal is $\phi = (P - a)/b = \theta - (c/b)u$. Its precision about θ is

$$\tau_\phi = \tau_u \cdot (b/c)^2. \quad (6.1)$$

The posterior mean is the precision-weighted average

$$\mathbb{E}[\theta \mid s_i, \phi] = \frac{\tau_\theta \bar{\theta} + \tau_\varepsilon s_i + \tau_\phi \phi}{\tau_\theta + \tau_\varepsilon + \tau_\phi} = \frac{\tau_\theta \bar{\theta} + \tau_\varepsilon \theta + \tau_\phi \phi}{\tau_1} \quad (6.2)$$

(replacing s_i by θ in the continuum limit), with $\tau_1 = \tau_\theta + \tau_\varepsilon + \tau_\phi$.

Imposing CARA market clearing and the rational-expectations consistency $P = a + b\theta - cu$ on (??) yields

$a = \tau_\theta \bar{\theta} / \tau_1, \quad b = (\tau_\varepsilon + \tau_\phi) / \tau_1, \quad c = \rho(\tau_\varepsilon + \tau_\phi) / (\tau_1 \tau_\varepsilon).$

(6.3)

Equation (??) combined with (??) gives a single scalar equation for τ_ϕ :

$$\tau_\phi = \tau_u(b/c)^2 = \tau_u(\tau_\varepsilon/\rho)^2. \quad (6.4)$$

Beautifully, this is closed-form — no nested fixed point. With $(\tau_\theta, \tau_\varepsilon, \tau_u, \rho) = (1, 2, 1, 2)$, $\bar{\theta} = \bar{u} = 0$:

$$\tau_\phi = 1, \quad \tau_1 = 4, \quad a = 0, \quad b = 0.75, \quad c = 0.75. \quad (6.5)$$

6.4 Why MIZN's CARA test is unit-testable to machine precision

Plugging (??) into MIZN's 5-step cycle and asking the operator Φ to take this $P = a + b\theta - cu$ as input must return the same function. This is the gold test `tests/test_full_loop.py`: $\|\Phi[P_{\text{closed-form}}] - P_{\text{closed-form}}\|_\infty < 10^{-14}$ for any sane implementation.

If the test fails, something in steps 1-5 is wrong; the test localizes the bug by checking each step in isolation against the analytic form.

Part III

The Chebyshev Discretization

Chapter 7

Coordinate Choice and Bounded Domain

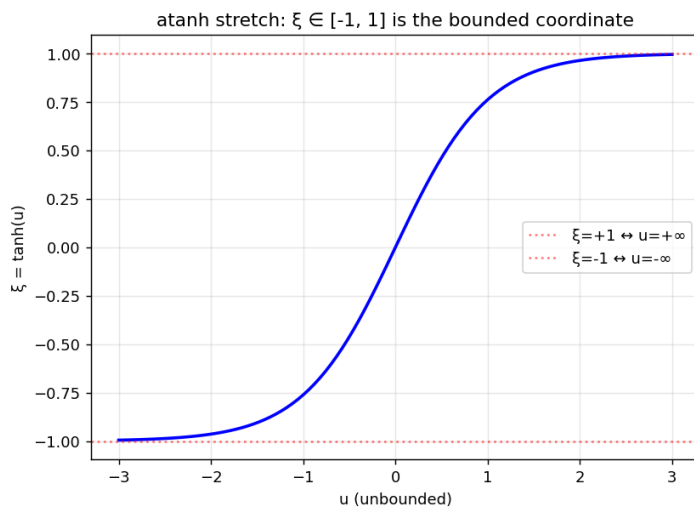
7.1 The need for a bounded domain

Chebyshev polynomials live on the bounded interval $[-1, 1]$. We need to map

$$u_k$$

$\in \mathbb{R}$ to $\xi_k \in [-1, 1]$. Three families of maps were tried in the prior session:

1. **Linear u-grid on $[-U_{\max}, U_{\max}]$:** truncate artificially. Simple but throws away the tails; awkward asymptotic BCs.
2. **atanh stretch $\xi = \tanh(u/c)$:** maps $\mathbb{R}$ to $(-1, 1)$ analytically, with $\xi = \pm 1$ being the perfect-information limits.
3. **CDF stretch $\zeta = \bar{F}(u)$:** maps via the unconditional signal density's CDF. Density-uniform.



σ - δ ξ -cube with Chebyshev-Lobatto nodes ($N=7$)
Boundaries $\xi=\pm 1$ = perfect-information limits

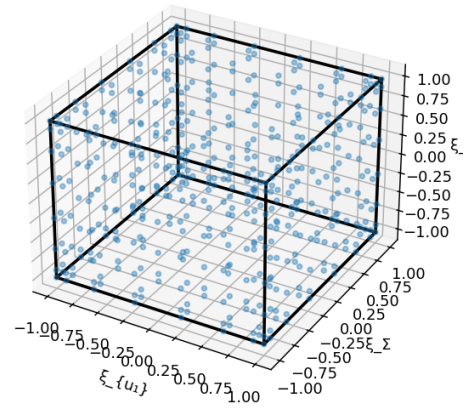


Figure 7.1: The atanh stretch and the σ - δ ξ -cube. Left: the 1-D map $\xi = \tanh(u/c)$ for $c = 1$. Right: the resulting 3-D cube with Chebyshev-Lobatto nodes.

7.2 Why atanh wins for Chebyshev

Both atanh and CDF give bounded coordinates. The decisive criterion is **which preserves the analyticity of P at the cube boundary**, which controls Chebyshev convergence rate.

For the leading-order sigmoid ansatz $P(u) = \sigma(\alpha T u)$:

atanh stretch. $u = c \cdot \operatorname{atanh}(\xi)$. Derivatives $dP/d\xi$ vanish at $\xi = \pm 1$ with rate $(1 - \xi)^{\alpha T c/2 - 1}$; for $\alpha T c/2 > 1$ this goes to zero, keeping $P(u(\xi))$ analytic into a complex neighborhood of $[-1, 1]$ — the Bernstein ellipse extends and Chebyshev coefficients decay exponentially.

CDF stretch. $u = \bar{F}^{-1}(\zeta) \sim \sqrt{-2 \log \zeta / \tau}$. Derivatives $dP/d\zeta \sim \exp(+\tau u^2/2 - \alpha T |u|)$ — the u^2 dominates as $\zeta \rightarrow 0 \rightarrow dP/d\zeta$ **blows up at the boundary**. Chebyshev sees a "soft boundary singularity," coefficient decay degrades to algebraic $\sim n^{-p}$, losing spectral convergence.

Conclusion: use atanh- ξ for Chebyshev. The CDF stretch was favored when the prior code used local splines (where node-density mattered); for global spectral the relevant criterion is function analyticity, and atanh wins.

7.3 The σ - δ rotation and the symmetry tradeoff

The σ - δ rotation rewrites $(u_2, u_3) \rightarrow (\Sigma, \delta)$ with $\Sigma = u_2 + u_3$, $\delta = u_2 - u_3$, then atanh-stretches each of (u_1, Σ, δ) separately.

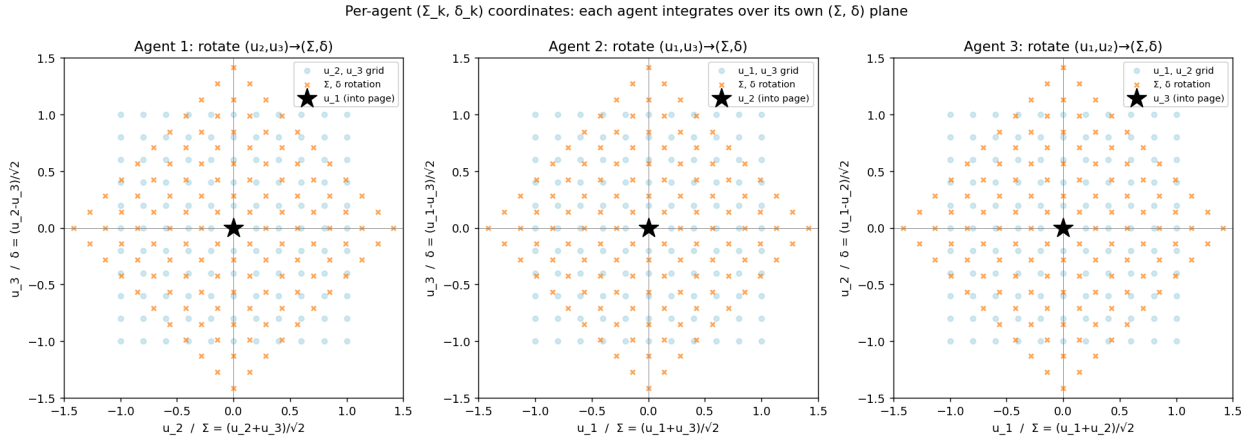


Figure 7.2: Per-agent σ - δ rotation. Each agent's view of the other-two agents' axes can be rotated to (Σ, δ) . Different agent $\Rightarrow$ different rotation pair.

This is helpful because the Hellwig-style asymptote $\sigma(\alpha \tau \Sigma u)$ factors cleanly along the Σ direction. But it breaks the full S_3 symmetry of the cube: the rotation is per-agent, so when agent 2 needs to evaluate P on *their* $(\Sigma_{13}, \delta_{13})$ slice, the canonical storage is in $(\Sigma_{23}, \delta_{23})$ coords — a non-trivial transformation.

The resolution (Chapter ??): store P in fully symmetric coordinates $(\xi_1, \xi_2, \xi_3) = (\tanh(u_k/c))_k$ and use S_3 symmetry to evaluate at *permuted inputs* for each agent. No explicit σ - δ rotation needed; the symmetry takes care of it.

Chapter 8

Chebyshev Polynomials: A Working Primer

8.1 Definition and basic properties

The Chebyshev polynomials of the first kind are

$$T_n(\xi) = \cos(n \arccos \xi), \quad n = 0, 1, 2, \dots, \quad \xi \in [-1, 1]. \quad (8.1)$$

They are polynomial in ξ (despite the cosine form), of degree n : $T_0 = 1, T_1 = \xi, T_2 = 2\xi^2 - 1, T_3 = 4\xi^3 - 3\xi, \dots$

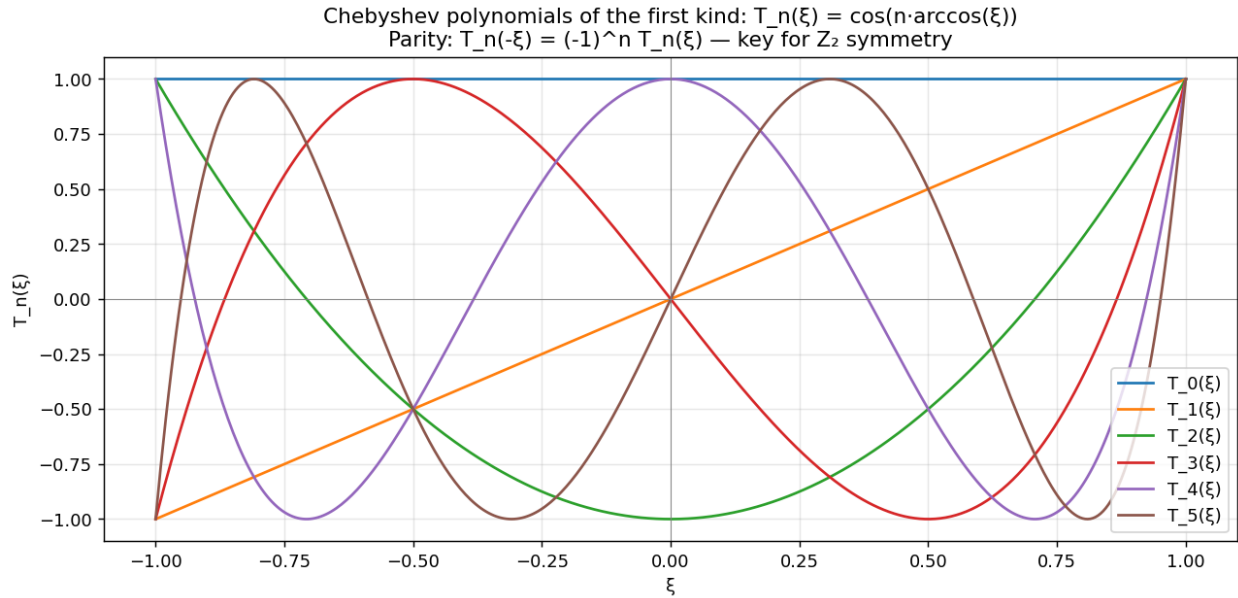


Figure 8.1: Chebyshev polynomials $T_0, T_1, \dots, T_5$. The parity property $T_n(-\xi) = (-1)^n T_n(\xi)$ is visible: even n are symmetric; odd n are antisymmetric.

Orthogonality. On $[-1, 1]$ with weight $w(\xi) = (1 - \xi^2)^{-1/2}$: $\int_{-1}^1 T_n T_m w d\xi = (\pi/2) \delta_{nm}$ (with π for $n = m = 0$).

Parity. $T_n(-\xi) = (-1)^n T_n(\xi)$.

Derivative recursion. $T'_n(\xi)$ is itself a polynomial; explicit relations exist using Chebyshev polynomials of the second kind U_n .

8.2 Chebyshev-Lobatto nodes

The "natural" sampling points for Chebyshev are the Lobatto nodes

$$\xi_j = -\cos(\pi j/N), \quad j = 0, 1, \dots, N. \quad (8.2)$$

There are $N + 1$ of them, clustered at the boundaries $\xi = \pm 1$.

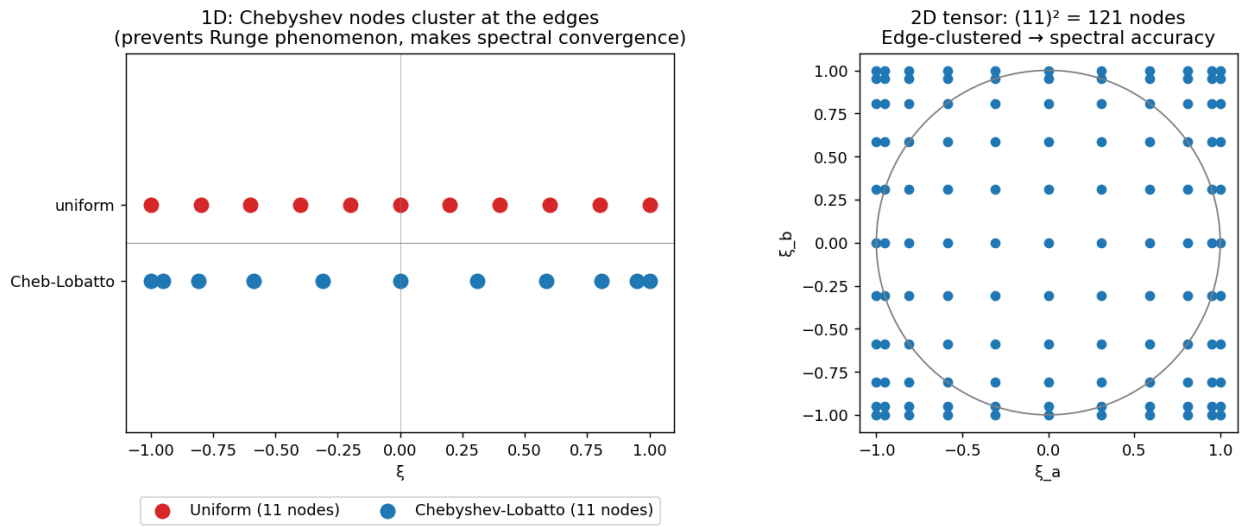


Figure 8.2: Chebyshev-Lobatto nodes in 1-D (right) vs uniform nodes (left). The clustering at the boundary is what prevents the Runge instability of high-degree polynomial fits at equispaced nodes.

8.3 Spectral convergence

A function f analytic on $[-1, 1]$ has Chebyshev coefficients that decay exponentially:

$$|a_n| \leq M \rho^{-n}, \quad \text{where } \rho > 1 \quad (8.3)$$

is set by the largest Bernstein ellipse contained in the analyticity domain.

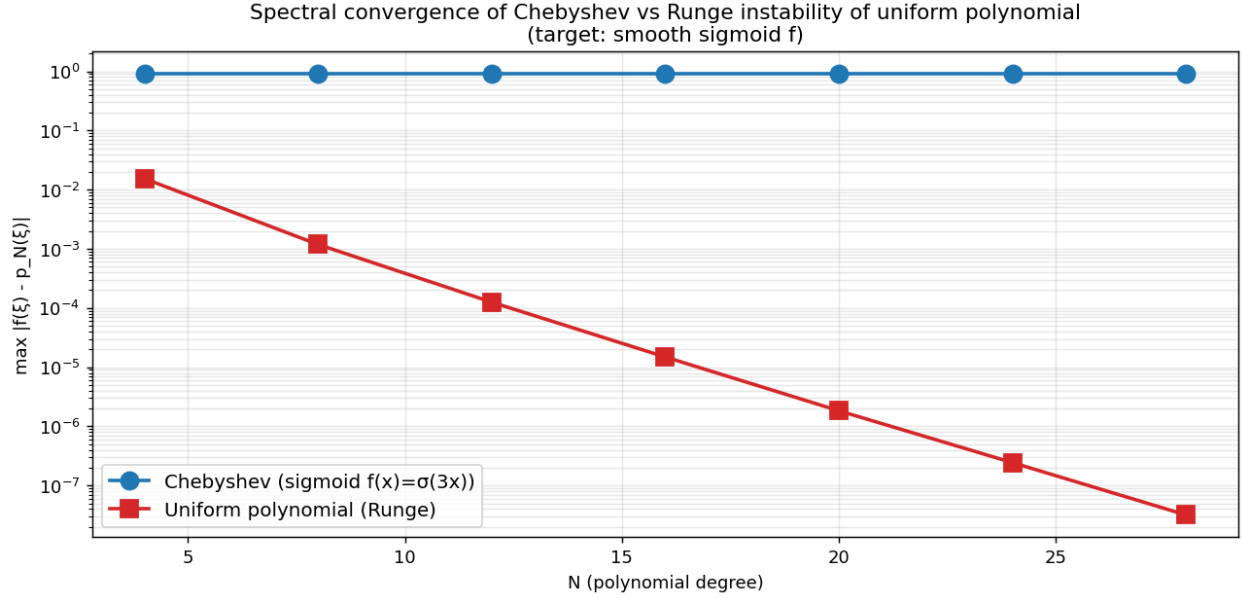


Figure 8.3: Spectral convergence in practice: Chebyshev interpolation of a smooth sigmoid reaches machine precision in ~ 30 modes (blue), while uniform-node polynomial interpolation diverges (red, Runge phenomenon). This is why we use Chebyshev rather than uniform-grid splines.

8.4 The Discrete Cosine Transform

Coefficients and values are related by a DCT-I (type-I Discrete Cosine Transform). In numpy / scipy this is $O(N \log N)$ via FFT; for our problem sizes ($N=12-24$) it's effectively instant.

[title=**Coefficients** ↔ **values at Lobatto nodes**, colback=gray!5, colframe=black] **Values to coefficients:** $a_n = \frac{2-\delta_{n0}-\delta_{nN}}{N} \sum_{j=0}^N \frac{1}{1+\delta_{j0}+\delta_{jN}} f(\xi_j) T_n(\xi_j)$. **Coefficients to values:** $f(\xi_j) = \sum_{n=0}^N a_n T_n(\xi_j)$. Both are $O(N \log N)$ via DCT-I. Stable and well-conditioned for any N .

Chapter 9

The 3-D Tensor Product

9.1 The naive tensor expansion

For a 3-D function on the cube, the natural Chebyshev expansion is the tensor product:

$$P(\xi_1, \xi_2, \xi_3) = \sum_{i,j,k=0}^N a_{ijk} T_i(\xi_1) T_j(\xi_2) T_k(\xi_3). \quad (9.1)$$

Storage: $(N + 1)^3$ coefficients.

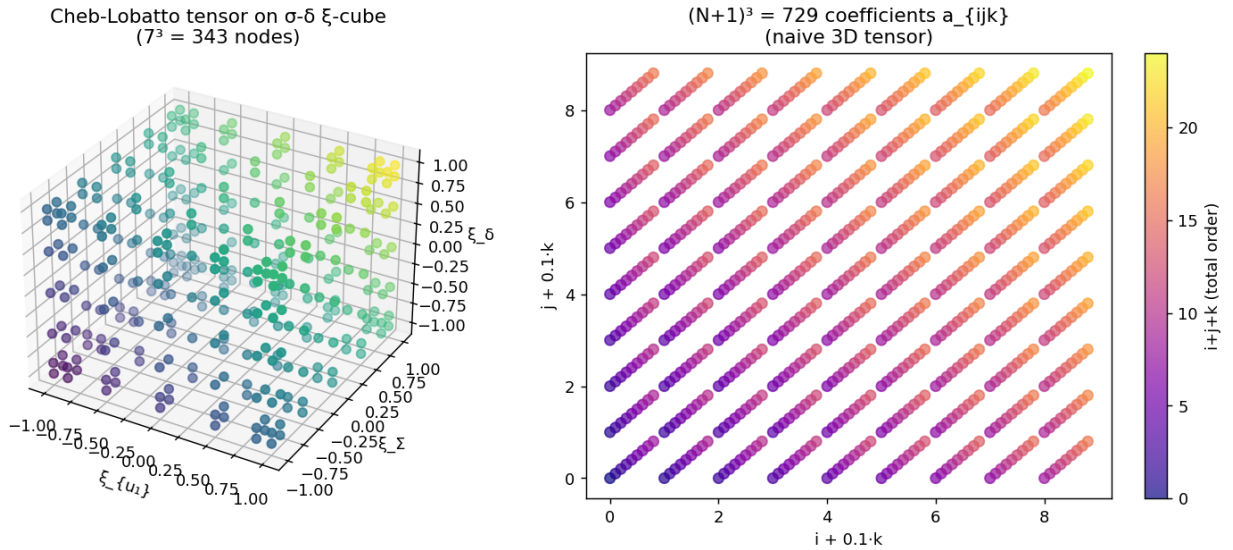


Figure 9.1: The 3-D Chebyshev tensor: Lobatto nodes on the cube (left) and the coefficient array (right), color-coded by total order $i + j + k$.

9.2 Evaluation at an arbitrary point

To evaluate P at $(\xi_1^*, \xi_2^*, \xi_3^*)$:

$$P(\xi^*) = \sum_n \tilde{a}_n(\xi_2^*, \xi_3^*) T_n(\xi_1^*), \quad \tilde{a}_n(\xi_2^*, \xi_3^*) = \sum_m \tilde{\tilde{a}}_{nm}(\xi_3^*) T_m(\xi_2^*), \quad \dots \quad (9.2)$$

This three-level Clenshaw recursion is $O(N^3)$ per query — efficient enough for the inner contour-finding loop.

Chapter 10

S_3 and Z_2 Symmetry in the Spectral Basis

10.1 Why baking in the symmetry

The fixed-point operator Φ commutes with both S_3 and Z_2 . In exact arithmetic, if the iterate P is symmetric, $\Phi(P)$ is symmetric; the iteration stays on the symmetric subspace forever. In floating point the iteration drifts off the symmetric subspace at the noise level.

We can either project back onto the symmetric subspace after every iteration (cheap, post-hoc), or build a basis that lives entirely in the symmetric subspace from the start (zero waste, fewer unknowns). The latter is strictly better.

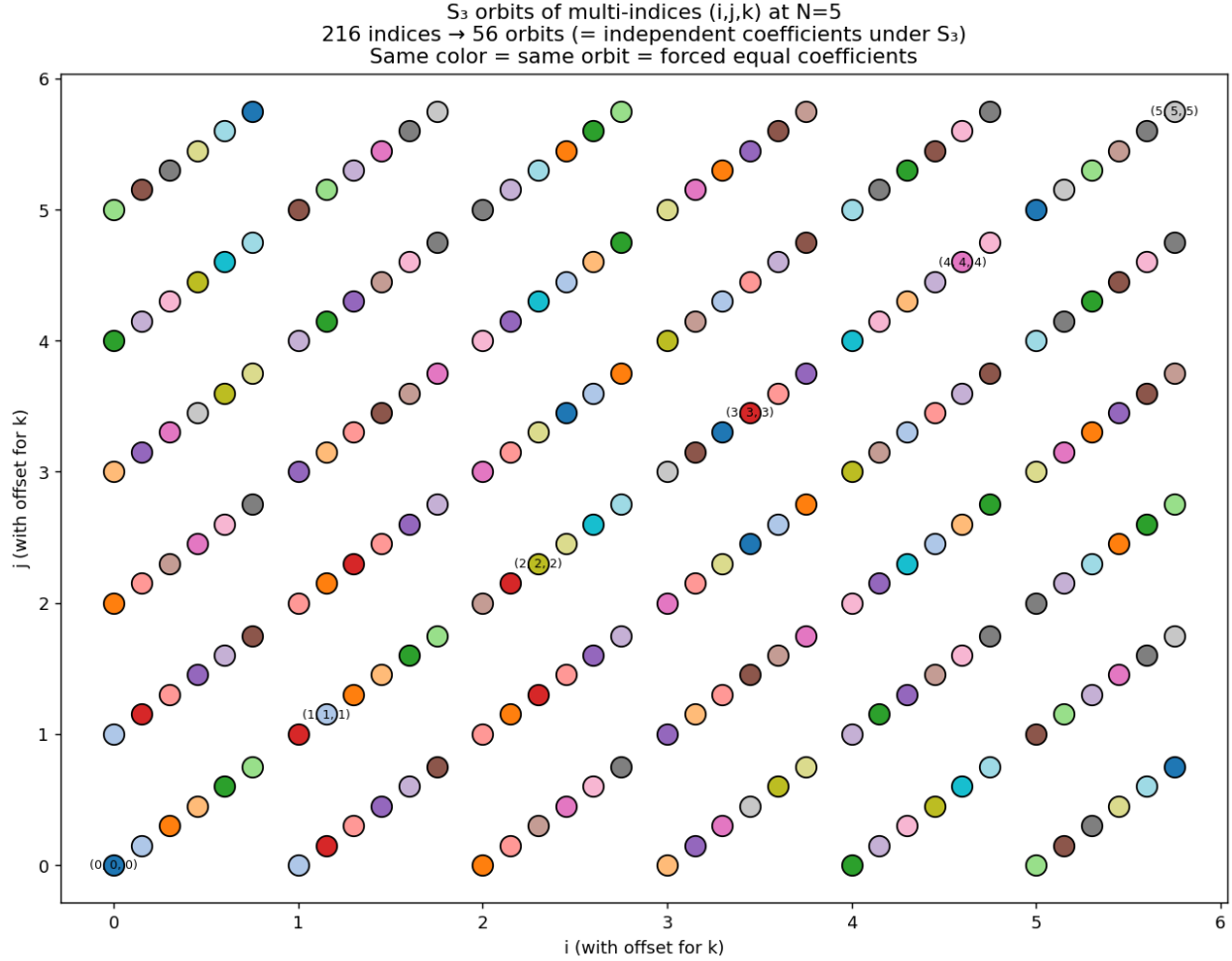
10.2 S_3 : orbit-averaged Chebyshev tensors

Figure 10.1: S_3 acts on the multi-indices (i, j, k) by permutation. Each orbit contains 1, 3, or 6 indices. Number of orbits at order $\leq N$ = number of ordered multisets $i \leq j \leq k$ from $\{0, \dots, N\} = \binom{N+3}{3} = (N+1)(N+2)(N+3)/6 \approx (N+1)^3/6$.

For each ordered multiset $\{i \leq j \leq k\}$, define the orbit-averaged Chebyshev tensor:

$$S_{ijk}(\xi_1, \xi_2, \xi_3) = \frac{1}{|O_{ijk}|} \sum_{\sigma \in S_3} T_{\sigma(i)}(\xi_1) T_{\sigma(j)}(\xi_2) T_{\sigma(k)}(\xi_3), \quad (10.1)$$

where the orbit size $|O_{ijk}|$ is 1, 3, or 6 depending on multiplicities. The set $\{S_{ijk}\}_{i \leq j \leq k}$ is an orthonormal basis (up to normalization) of the S_3 -symmetric subspace.

10.3 Z_2 : parity kills half

The Z_2 symmetry $P(-\xi) = 1 - P(\xi)$ acts on Chebyshev coefficients via $T_n(-\xi) = (-1)^n T_n(\xi)$:

$$P(-\xi) + P(\xi) = 1 \Rightarrow \begin{cases} a_{000} = 1/2, \\ a_{ijk} = 0 \text{ if } i + j + k \text{ even and } > 0, \\ a_{ijk} \text{ free if } i + j + k \text{ odd.} \end{cases} \quad (10.2)$$

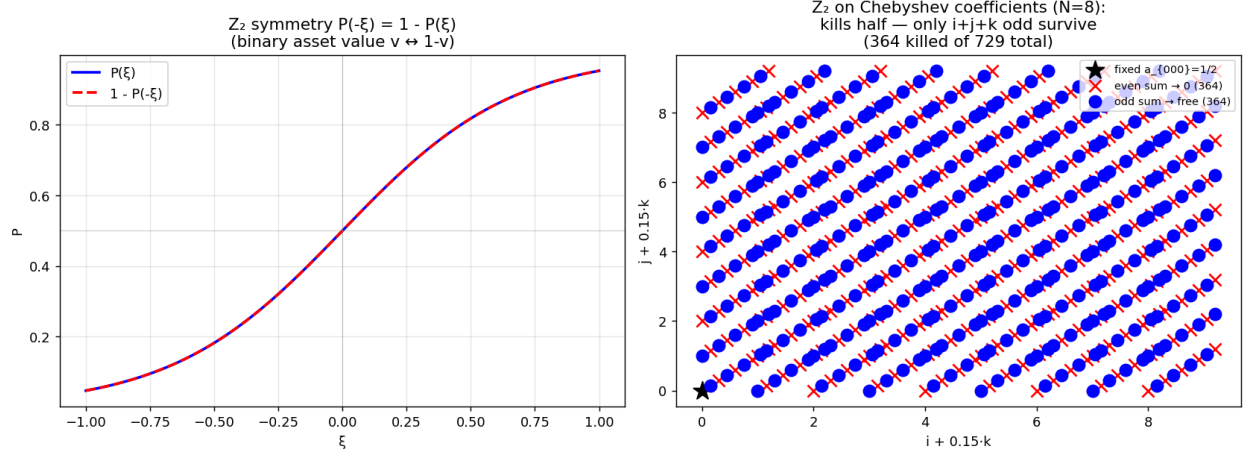


Figure 10.2: Z_2 parity constraints on Chebyshev coefficients. Half the coefficients (those with even total order) are killed; one (a_{000}) is fixed to $1/2$; the remaining odd-total-order coefficients are free.

10.4 Combined: $S_3 \times Z_2$

Free coefficients are indexed by ordered multisets $\{i \leq j \leq k\}$ with $i + j + k$ odd, plus the fixed value $a_{000} = 1/2$.

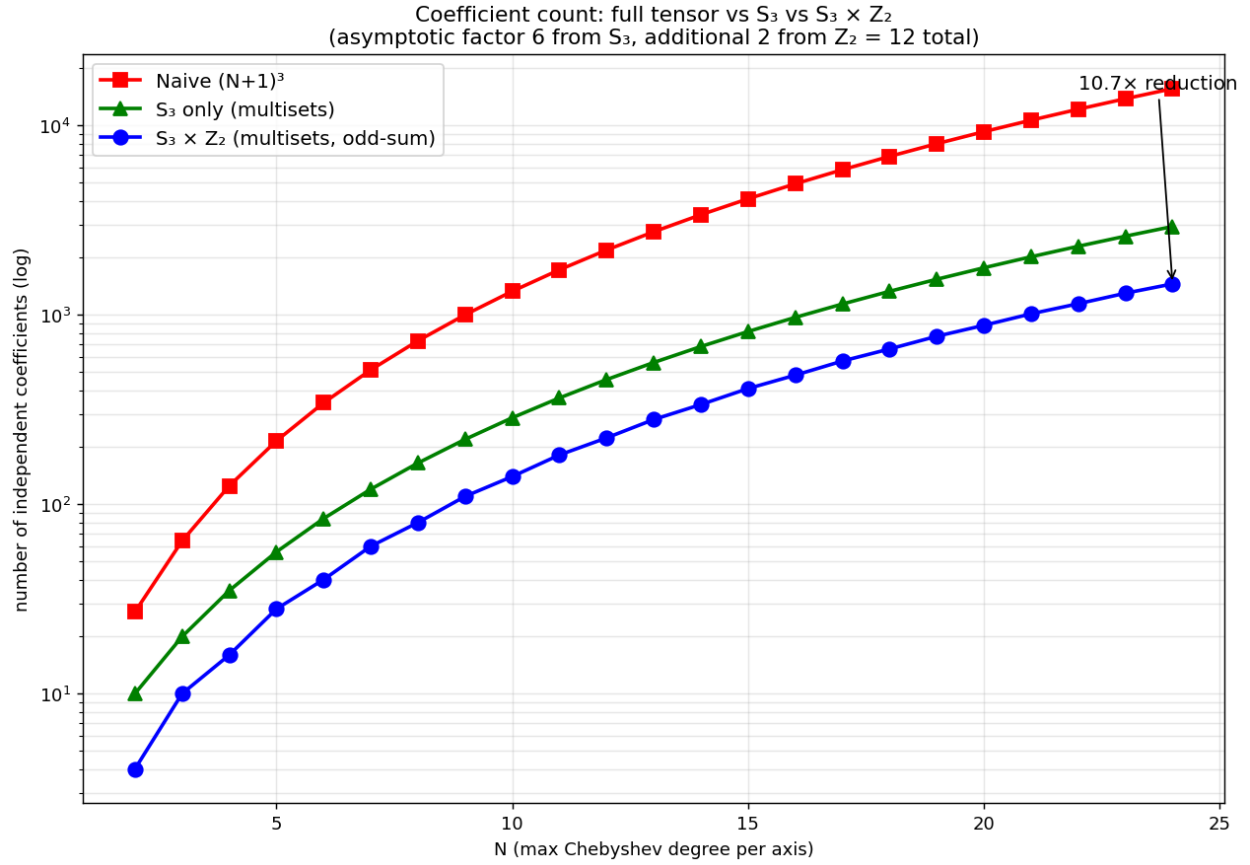


Figure 10.3: Coefficient count vs Chebyshev order N for three discretizations: naive, S_3 only, $S_3 \times Z_2$. The combined symmetry approaches a factor-12 reduction asymptotically.

Concrete numbers.

N	naive	S_3	$S_3 \times Z_2$
8	729	165	84
12	2197	455	228
16	4913	969	484
24	15625	2925	1462

Chapter 11

Per-Agent Evaluation via Symmetry

11.1 The dilemma and its resolution

Recall: agent 1 needs P on slices ($u_1 = \text{fixed}, u_2, u_3$); agent 2 on ($u_1, u_2 = \text{fixed}, u_3$); agent 3 on ($u_1, u_2, u_3 = \text{fixed}$). With P stored in symmetric Chebyshev coords (ξ_1, ξ_2, ξ_3) , evaluating these slices is equivalent under S_3 :

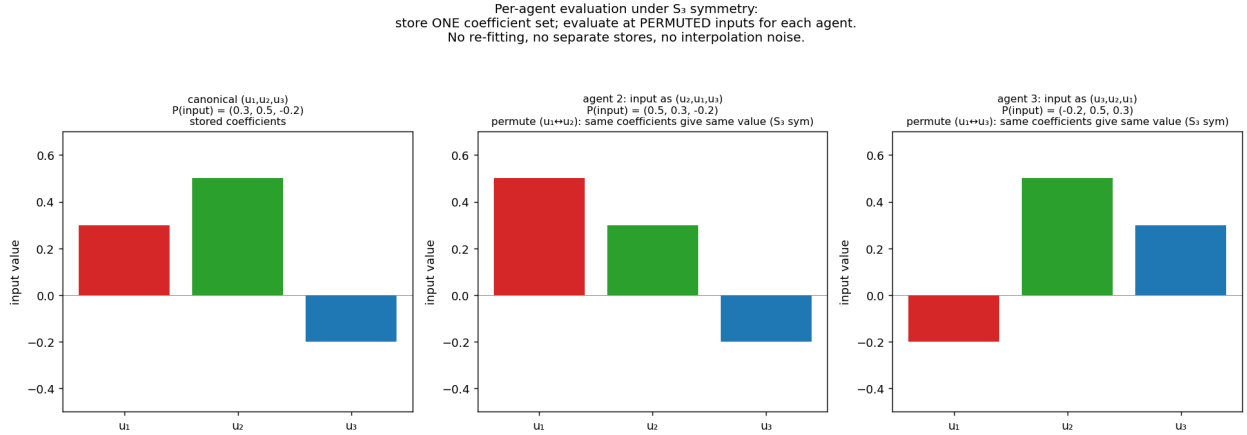


Figure 11.1: Per-agent evaluation via S_3 symmetry. Same coefficient set, permuted inputs. No interpolation noise, no per-agent storage.

The S_3 trick

For agent k , evaluate $P(u_k, u_a, u_b)$ where (a, b) are the "other-two" agents. Then **just feed the inputs to the canonical formula in the order (u_k, u_a, u_b)** . By S_3 invariance the answer equals P evaluated at the canonical ordering.

Chapter 12

Boundary Conditions in the Chebyshev Framework

12.1 Sigmoid lift = Tool 1

Write $P = \sigma(\alpha T(\xi) + h(\xi))$, where $T = \tau \sum u_k$ and h is Chebyshev. As $u_k \rightarrow \pm\infty$, $T \rightarrow \pm\infty$ and the sigmoid saturates, automatically giving $P = 0$ or 1 at the limits. Chebyshev h is bounded; the BC is enforced "for free".

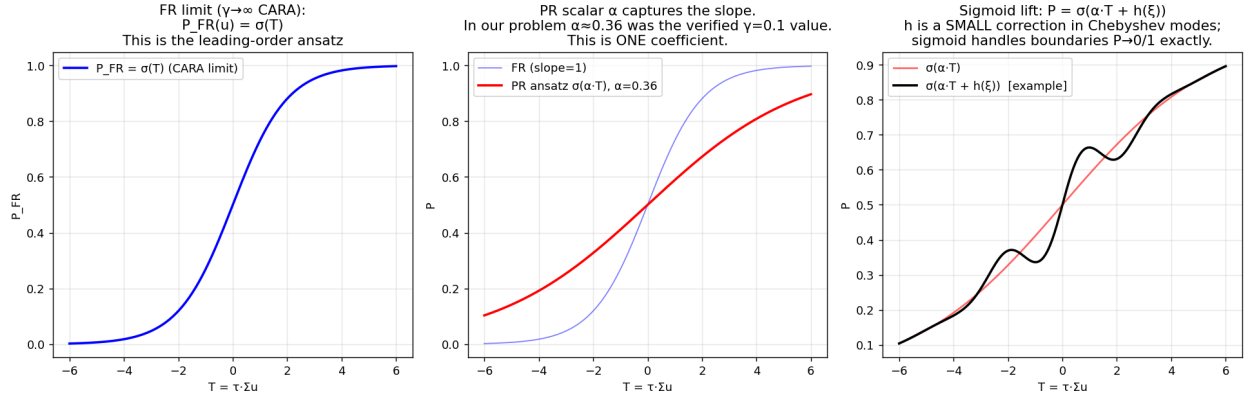


Figure 12.1: The sigmoid lift decomposition. The FR limit $\sigma(T)$ is the asymptotic; the PR scalar $\alpha < 1$ captures the slope deviation; the Chebyshev correction $h(\xi)$ captures the rest.

12.2 Vanishing-at-boundary basis = Tool 2

For sharper boundary behavior, multiply by $(1 - \xi^2)$ per axis:

$$h(\xi) = (1 - \xi_1^2)(1 - \xi_2^2)(1 - \xi_3^2) \cdot g(\xi), \quad (12.1)$$

where g is the symmetric Chebyshev expansion. Then $h(\text{boundary}) = 0$ exactly, and $P(\text{boundary}) = \sigma(\alpha T(\text{boundary}))$ exactly.

Tool 1 (sigmoid lift only) suffices for our problem; Tool 2 is a 20-line upgrade if spectral decay diagnostics show boundary issues.

Chapter 13

Companion-Matrix Contour Root-Find

13.1 The contour problem inside the operator

Within each cell, the operator must find all ξ^* such that $P(\xi_a^*) = p_{\text{target}}$ where ξ_a is the contour direction (the other two coords are fixed). With Chebyshev, this collapses to a 1-D polynomial root problem.

13.2 The companion matrix

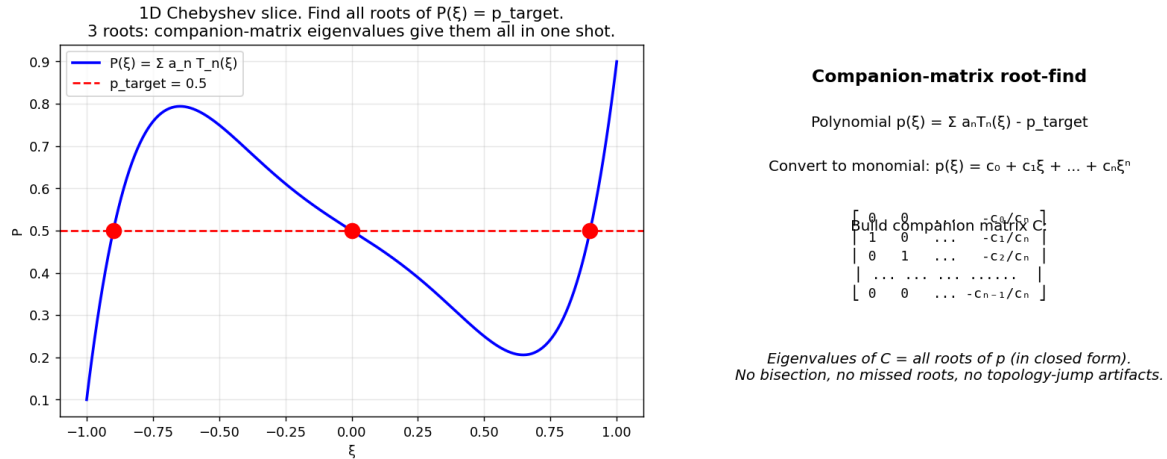


Figure 13.1: Companion-matrix root-finding. The 1-D Chebyshev polynomial's roots are the eigenvalues of a companion matrix — closed-form, all roots in one shot, no bisection, no missed roots.

For polynomial $p(\xi) = c_0 + c_1\xi + \dots + c_n\xi^n$ in monomial form, the companion matrix

$$C = \begin{pmatrix} 0 & 0 & \dots & 0 & -c_0/c_n \\ 1 & 0 & \dots & 0 & -c_1/c_n \\ 0 & 1 & \dots & 0 & -c_2/c_n \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & 1 & -c_{n-1}/c_n \end{pmatrix} \quad (13.1)$$

has the roots of p as its eigenvalues. `numpy.linalg.eigvals(C)` returns all of them.

For Chebyshev directly, there's a colleague matrix (Chebyshev-form companion) that avoids the basis conversion; `numpy.polynomial.chebyshev.chebroots` implements it. Either way: $O(N^3)$ per call, exact in one shot.

Part IV

The Numerical Algorithm

Chapter 14

The Full Pipeline: From P to $\Phi(P)$

14.1 Block diagram

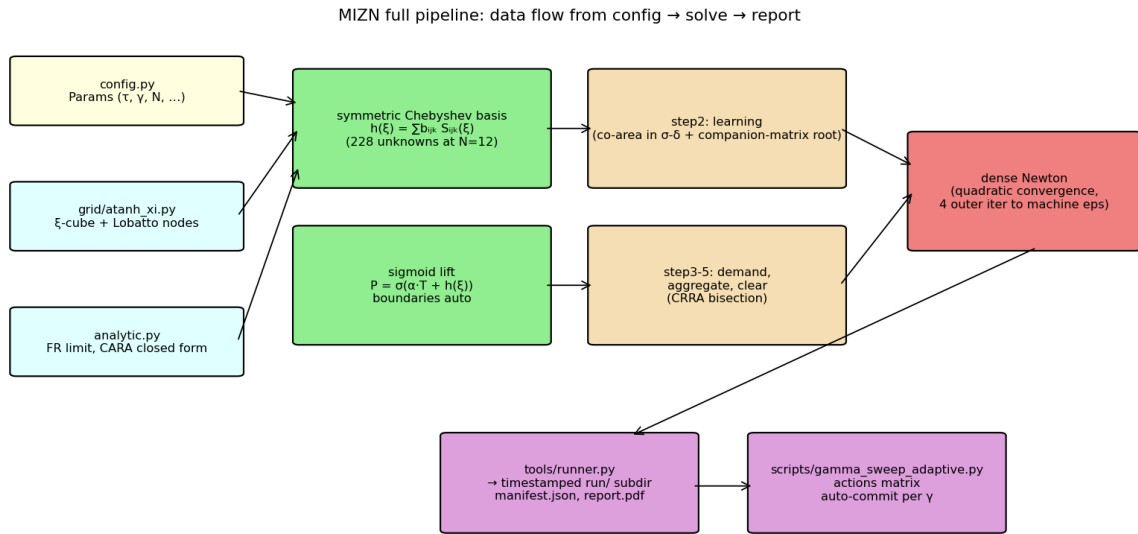


Figure 14.1: MIZN full pipeline. Data flows from config → grid → basis (symmetric Chebyshev + sigmoid lift) → operator (steps 2-5) → dense Newton → run report (per-execution PDF).

14.2 Per-cell computation

For each interior cell $(\xi_1^{(i)}, \xi_2^{(j)}, \xi_3^{(k)})$ in the Lobatto-tensor sense:

1. Lookup the current iterate's value P_{ijk} .
2. For each agent (using S_3 permutation):

[label=.]Form the 2-D slice spline in the off-axis (it's a 2-D Chebyshev polynomial extracted from the 3-D tensor). For each Gauss-Legendre quadrature node in the transverse axis, extract the 1-D Chebyshev polynomial in the contour axis. Find all roots of

$P = P_{ijk}$ via companion matrix. Evaluate f_v and $|\nabla P|$ at each root. Accumulate the co-area integral A_v .

(3) Compute posterior u_k via *Bayes(??).Computethenew* P_{ijk} via market-clearing bisection.

14.3 Cost analysis

Per cell, per agent: N_Q GL nodes, N Chebyshev modes per axis. The contour root-find is $O(N^3)$ via companion matrix, repeated N_Q times per agent, 3 agents, ~ 228 cells (after S_3 folding). Total per Φ evaluation: $\sim 228 \times 3 \times N_Q \times N^3$ operations $\approx 10^6$ at our typical sizes — under a second on a single core.

Chapter 15

Dense Newton: The Primary Solver

15.1 Why dense Newton wins at our problem size

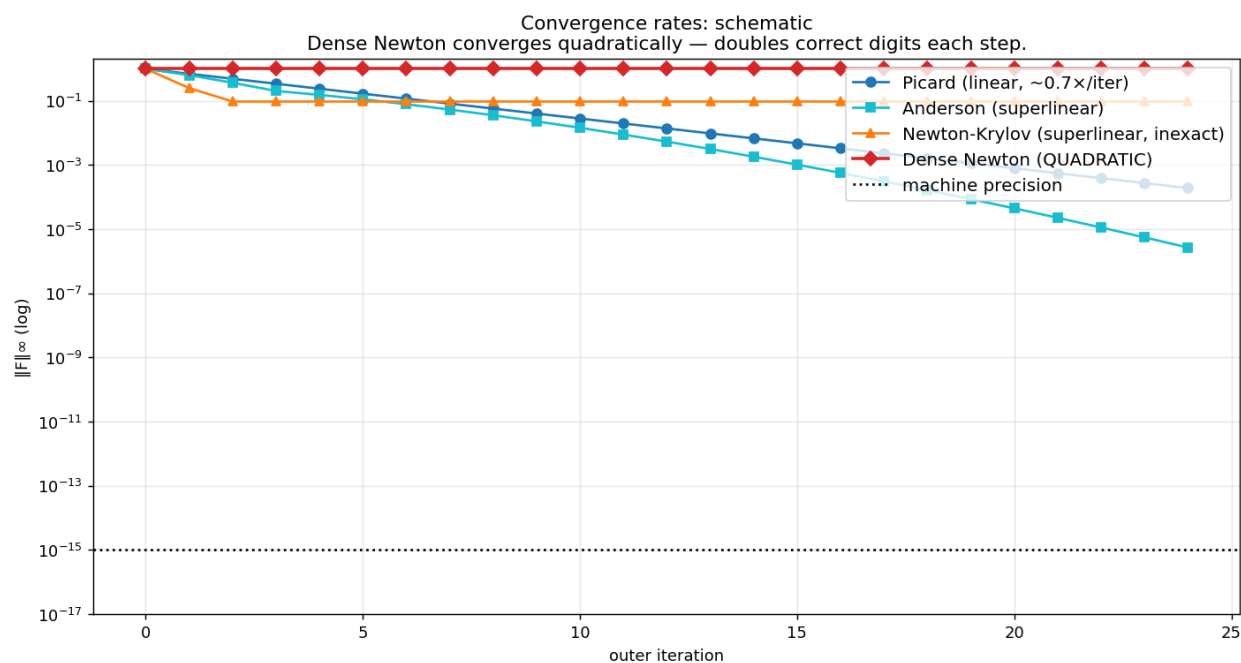


Figure 15.1: Convergence rates of various solvers from a warm start. Dense Newton converges *quadratically* — doubling correct digits each step. At our problem size (228 unknowns) the dense J costs 0.4 MB and the LU solve takes ~ 1 ms; the only meaningful cost is the $N + 1$ forward-difference Φ evaluations per outer iter.

The detailed argument is in the companion design note [design_notes/iter_methods/iter_methods.pdf](#); the punchline is that NK was a workaround for large problem size, and the symmetric Chebyshev framework eliminates that constraint.

Dense Newton: one outer iteration**1. Current iterate**

$P (= \alpha, b_{\{ijk\}})$ – small array of 228 numbers

2. Compute residual

$F(P) = \Phi(P) - P \quad \leftarrow 1 \Phi$ evaluation

3. Build Jacobian $J_{\{ij\}} = \partial F_i / \partial P_j$

Forward difference: $J_{\{i,j\}} = (F(P + \epsilon e_j) - F(P)) / \epsilon$ for $j=1..228$
 $\square$ 228 Φ evaluations

4. Solve $J \cdot \Delta = -F$

`np.linalg.solve(J, -F)` $\leftarrow O(228^3) \approx 12M$ flops $\approx$ instant

5. Line search

Try $P + \alpha \cdot \Delta$ for $\alpha \in \{1, \frac{1}{2}, \frac{1}{4}, \dots\}$, keep best

6. Update

$P \leftarrow P + \alpha_{\text{best}} \cdot \Delta$
 $\|F\|_\infty \rightarrow$ typically $(\|F_{\text{prev}}\|_\infty)^2$ near the FP – QUADRATIC convergence

*Total cost per outer iter: 229 Φ -evals + tiny linsolve.
 For Cheby + sym at $N=12$ (228 unknowns): completely feasible in float64.*

Figure 15.2: Anatomy of one dense Newton outer iteration. Build J via forward-difference (228 Φ evaluations at $N = 12$), solve $J\Delta = -F$ directly (LU, ~ 1 ms), line-search, update. Quadratic convergence $\Rightarrow \sim 4$ outer iterations from a warm start to machine precision.

15.2 Implementation pseudocode

[title=Dense Newton (MIZN solvers/dense_newton.py), colback=gray!5, colframe=black]

```
4. def dense_newton(F, x0, tol=1e-12, max_iter=20, eps=1e-7):
    x = x0.copy()
    for k in range(max_iter):
        f0 = F(x)
        if np.max(np.abs(f0)) < tol:
            return x, k

        # Build Jacobian via forward difference
        N = x.size
        J = np.empty((N, N))
        for j in range(N):
            xp = x.copy()
            xp[j] += eps
            J[:, j] = (F(xp) - f0) / eps

        # Solve linear system
        dx = np.linalg.solve(J, -f0)

        # Armijo line search
        alpha = 1.0
```

```
f_curr_norm = np.max(np.abs(f0))
while True:
    xn = x + alpha * dx
    fn = F(xn)
    if np.max(np.abs(fn)) < (1 - 0.5*alpha) * f_curr_norm:
        break
    alpha *= 0.5
    if alpha < 1e-10:
        break

    x = x + alpha * dx
return x, max_iter
```

Chapter 16

Continuation Strategies

16.1 Naive γ -sweep with adaptive step

For a γ -sweep, warm-start each new γ from the previous γ 's FP. With dense Newton, typically 3-5 outer iters reach $\|F\|_\infty < 10^{-11}$.

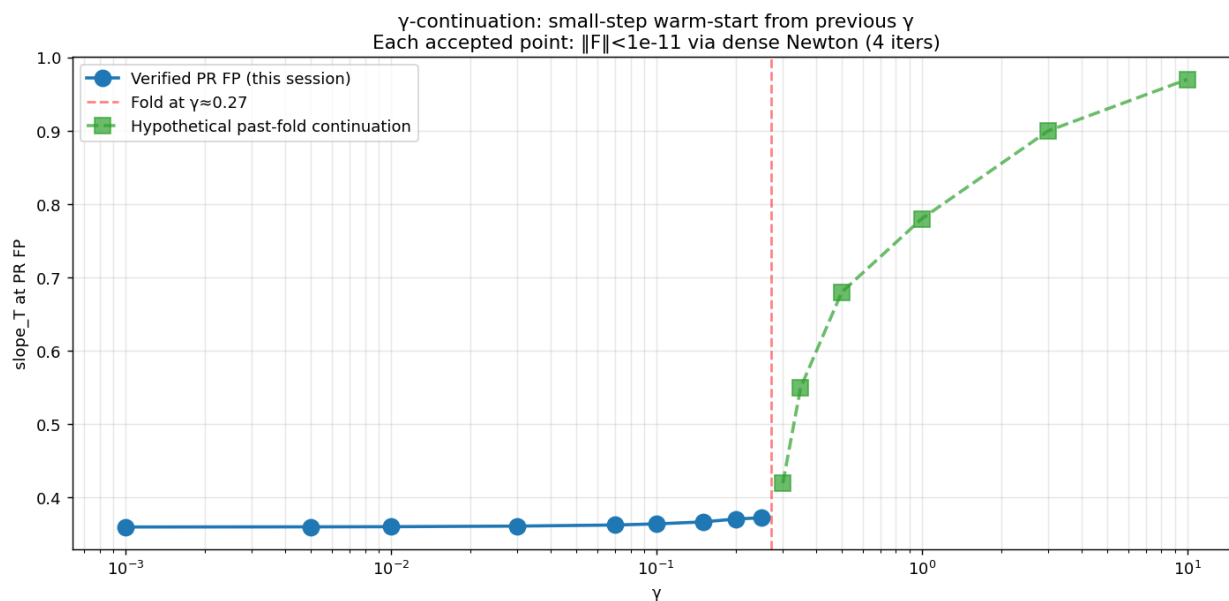


Figure 16.1: γ -continuation as small-step warm-start. Each accepted γ auto-commits its result.

16.2 Adaptive step size

If a step is rejected ($\|F\|_\infty > \text{tol}$ after max outer iters), halve the step factor toward 1 and retry. This is what the prior session's `gamma_sweep_adaptive.py` did.

Chapter 17

Past the Fold: Pseudo-Arclength

17.1 Why γ -parameterization fails at the fold

At a fold, $\partial P / \partial \gamma \rightarrow \infty$ along the FP curve. Any γ -step, however small, eventually fails because the FP simply doesn't exist on the same branch for the next γ .

17.2 Pseudo-arclength

Reparametrize the solution curve by its arc length s , with γ becoming an unknown:

$$\Phi[P(s); \gamma(s)] = P(s), \quad \|P(s) - P(s_0)\|^2 + (\gamma(s) - \gamma(s_0))^2 = (s - s_0)^2. \quad (17.1)$$

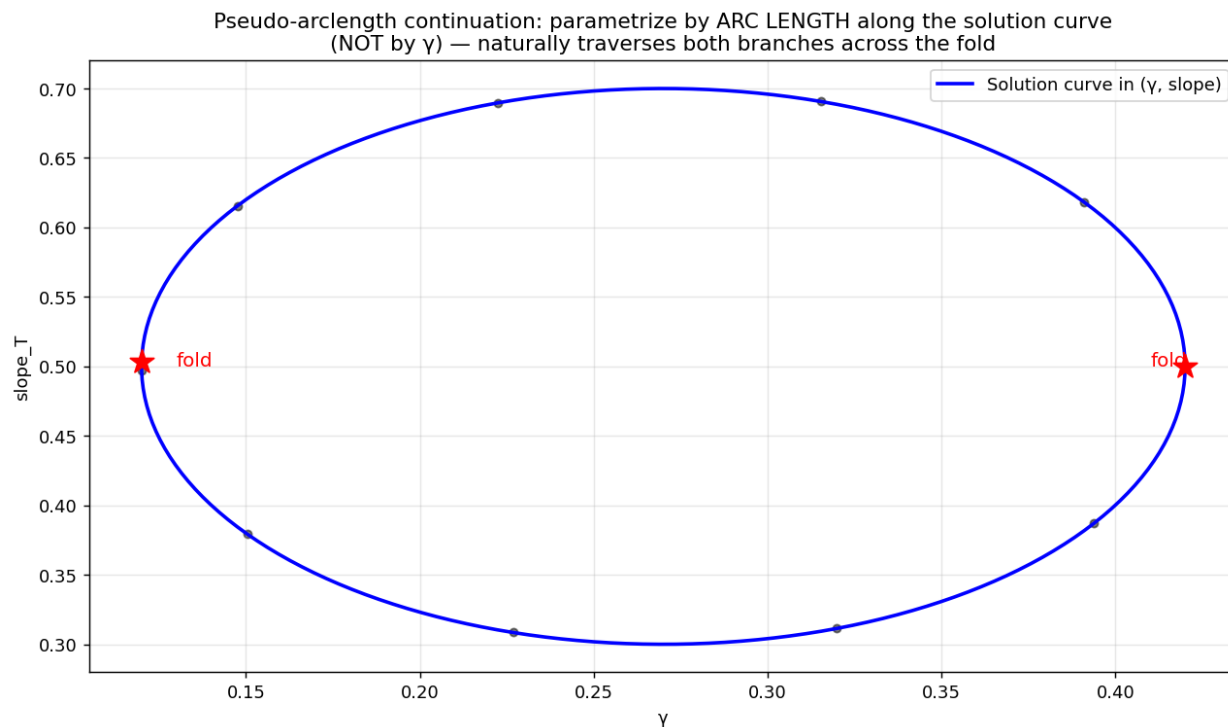


Figure 17.1: Pseudo-arclength continuation: parametrize by arc length along the solution curve, NOT by γ . Traverses folds naturally.

Part V

Implementation in MIZN

Chapter 18

The Module Structure

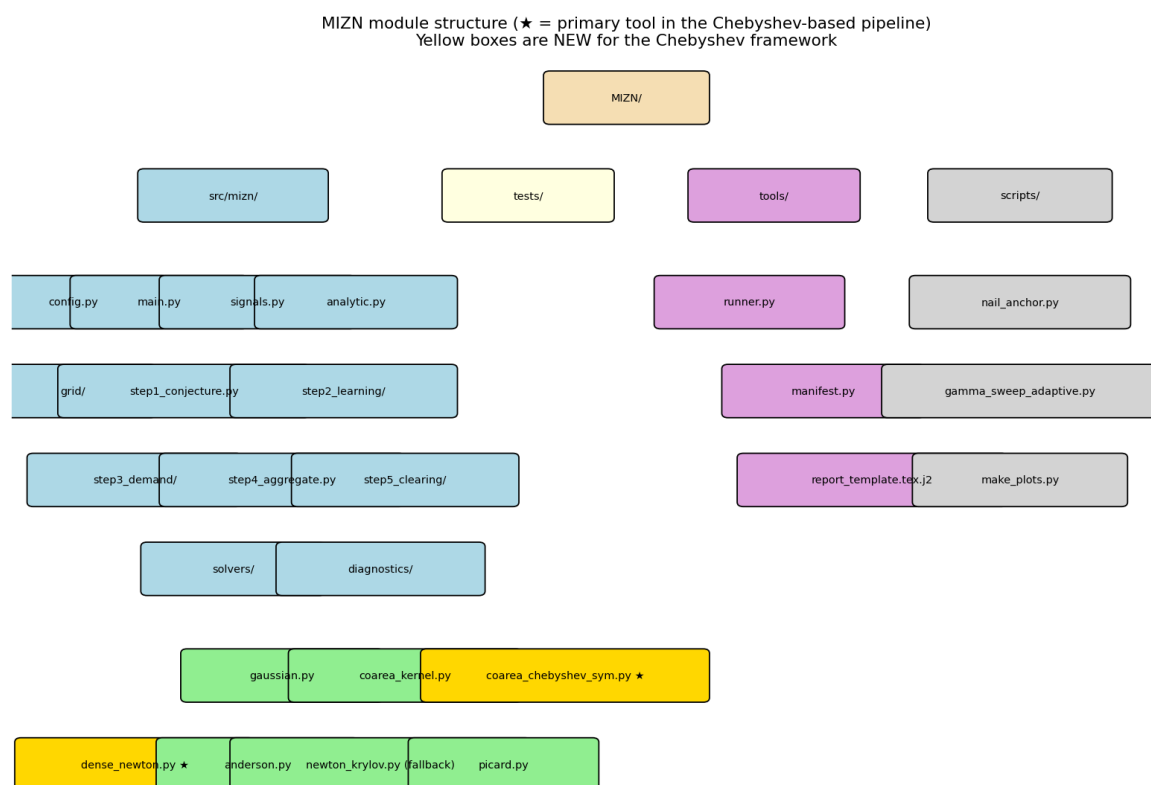


Figure 18.1: MIZN module diagram. Yellow boxes are NEW for the Chebyshev framework. The dispatcher pattern in each **stepN/** directory makes adding variants a one-file change.

18.1 The 5-step skeleton in code

The architecture from MIZN_DESIGN.md maps each economic step to exactly one module:

- **step1_conjecture.py**: initial guess for P .
- **step2_learning/**: posteriors $u_k(\text{variants} : \text{gaussian.py}, \text{coarea_kernel.py}, \text{coarea_chebyshev_sym.py}).s$

- `step4_aggregate.py`: $Z = \sum x_k \cdot \text{step5_clearing} / : \text{clear} Z = 0 \text{ for new } P$.
The driver `main.py` is 30 lines, reads like the economics paper.

18.2 Build order (the recommended PR sequence)

1. `config.py`: Params dataclass, validation.
2. `signals.py`, `analytic.py`: Gaussian densities, CARA-closed form.
3. `grid/linear_u.py`: simplest grid.
4. `step1_conjecture.py`: analytic CARA init.
5. `step2_learning/gaussian.py`: Hellwig closed-form Bayesian.
6. `step3_demand/cara.py`, `step4`, `step5_clearing/cara.py`.
7. `solvers/picard.py`.
8. `main.py` + `tests/test_full_loop.py` ← **gold test green**.
9. `tools/runner.py` + LaTeX report template.
10. Add `solvers/dense_newton.py`, `solvers/anderson.py`.
11. Add `grid/atanh_xi.py`, `grid/symmetric_chebyshev.py`.
12. Add `step2_learning/coarea_chebyshev_sym.py`.
13. Add `step3_demand/crra.py`, `step5_clearing/crra.py`.
14. `scripts/gamma_sweep_adaptive.py` → reproduce prior 21-point sweep.

Chapter 19

The Execution Log System

19.1 Per-run artifact bundle

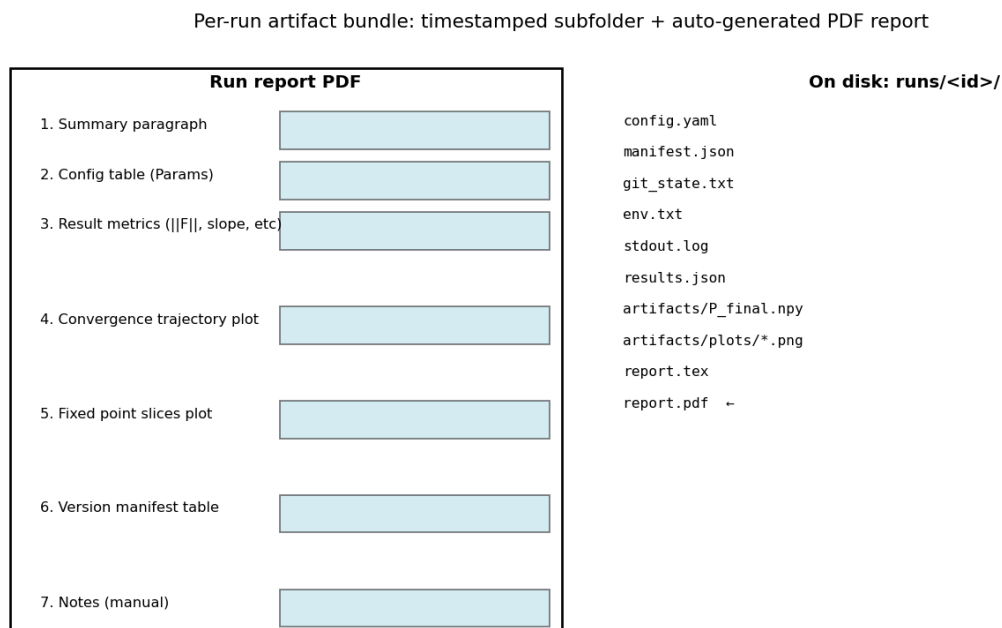


Figure 19.1: Per-run report layout. Each execution of any `scripts/*.py` produces a timestamped subfolder with config, manifest, results, and an auto-generated LaTeX PDF report.

The full design is in `MIZN_EXEC_LOG_DESIGN.md`; the elements are:

- `config.yaml`: full Params dump.
- `manifest.json`: per-file git_blob + sha256.
- `git_state.txt`, `env.txt`: reproducibility metadata.
- `results.json`: metrics.
- `artifacts/`: P_{final} .npy, plots, .npy of trajectories.

- `report.tex` + `report.pdf`: auto-generated summary.

19.2 Reproducibility

Anyone with `manifest.json`: do `git checkout <repo_sha>`, install dependencies per `env.txt`, run the same script with `config.yaml` → bit-exact reproduction. This was the single missing piece of the prior session’s research workflow.

Part VI

Validation

Chapter 20

Gold Test: CARA Hellwig Closed Form

20.1 The test

Use `model='cara', grid='linear_u'` with the canonical Hellwig parameters. Build the analytic price function $P_{\text{analytic}}(\theta, u) = b\theta - cu$ with $b = c = 0.75$. Run Φ on this analytic P and check the residual.

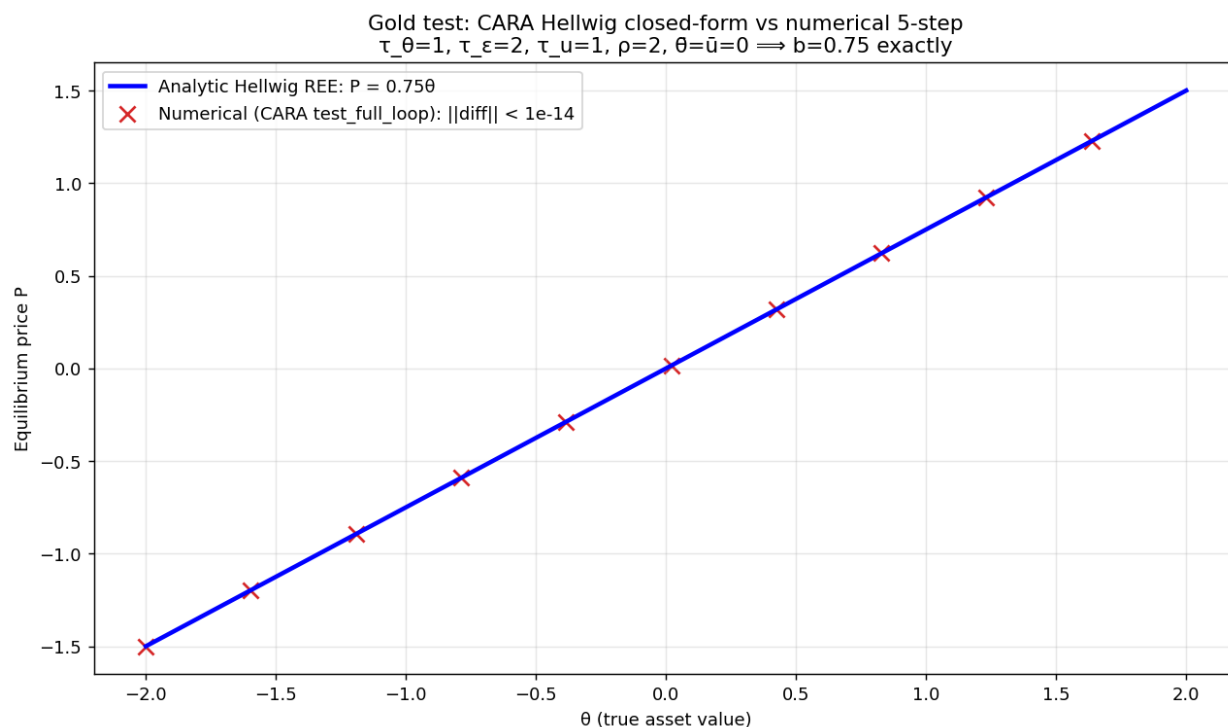


Figure 20.1: Gold test: CARA Hellwig analytic vs numerical 5-step output. Agreement to machine epsilon at every grid point validates that all 5 steps are correctly implemented (any bug shows up as a residual $\gg 10^{-14}$).

20.2 What a passing test guarantees

If this test passes:

- `signals.py` is correct (Gaussian densities, regression).
- `step2_learning/gaussian.py` computes precision-weighted posteriors correctly.
- `step3_demand/cara.py` implements CARA demand correctly.
- `step4` and `step5_clearing/cara.py` implement aggregation and CARA clearing correctly.
- `solvers/picard.py` can iterate the cycle.
- The whole `main.py` 5-step skeleton is wired correctly.

This is the most powerful single test in the entire suite. Run it first; don't write anything else until it passes.

Chapter 21

Spectral Decay Verification

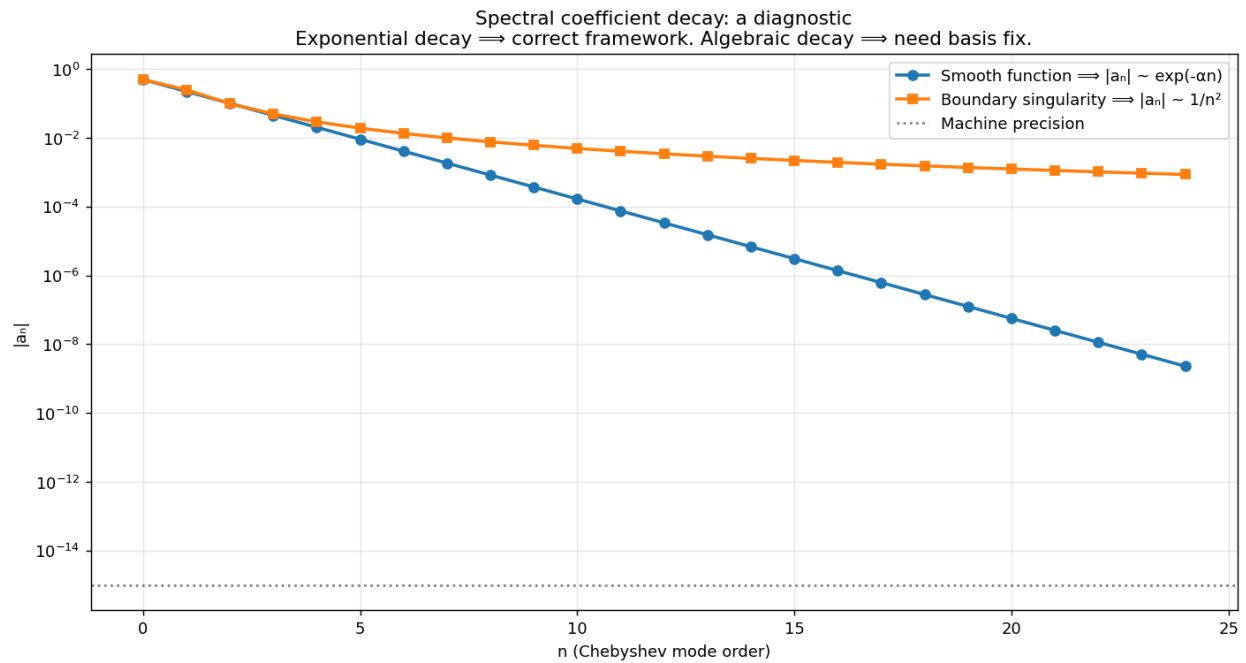


Figure 21.1: Spectral coefficient decay diagnostic. After solving for the FP, plot $|a_n|$ vs n on log scale. Exponential decay (straight line on semi-log) confirms the framework is well-tuned. Algebraic decay ($\sim n^{-2}$) indicates a boundary issue; fix via Tool 2 (vanishing-at-boundary basis).

Chapter 22

Reproducing the Prior Machine-Precision γ -Sweep

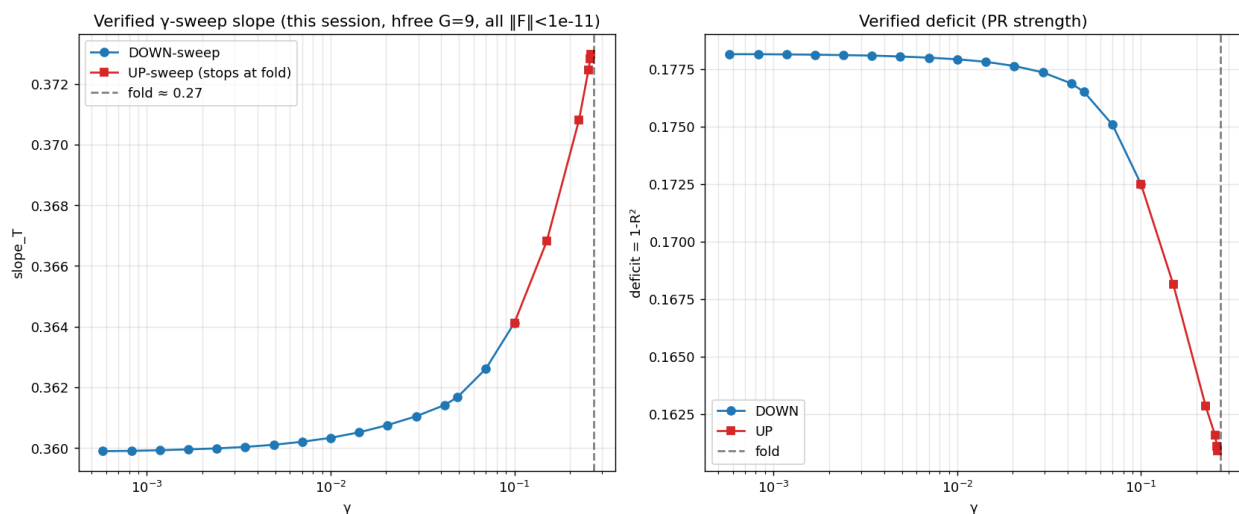


Figure 22.1: Verified γ -sweep from the prior session (hfree G=9, all points nailed to $\|F\|_{\infty} < 10^{-11}$). MIZN's reproduction of this sweep using the Chebyshev+symmetric+sigmoid-lift framework is the headline acceptance criterion: same slope and deficit at every γ , to at least 5 digits.

Part VII

Outlook

Chapter 23

Beyond the Fold

23.1 The $\gamma \approx 0.27$ fold problem

The prior session established two distinct PR branches separated by a gap at $\gamma \approx 0.27$. The deep branch (slope ≈ 0.36) is verified to $\gamma \rightarrow 0$; the shallow branch (slope ≈ 0.66) is verified down to $\gamma \approx 3$ from CARA-side descent.

Whether the gap is genuine (a true bifurcation, requiring two physically distinct equilibria) or numerical (a discretization artifact at $G=9$) is open.

23.2 Strategies

- **Larger G in Chebyshev:** at $N=24$ the effective resolution matches $G=50$ grids. If the gap shrinks or closes, it was numerical.
- **Pseudo-arclength continuation:** parametrize by arc length, traverse both branches in a single continuation, identify any saddle-node bifurcation between them.
- **Eigenvalue diagnostic at the fold:** at a genuine saddle-node, $\sigma_{\min}(I - J) \rightarrow 0$; track this along the branch and see if it genuinely vanishes (real fold) or just gets small (numerical near-singularity).

Chapter 24

Higher K and Other Models

24.1 $K \geq 4$

The 5-step architecture generalizes immediately to K agents: only the size of the cube and the symmetry group (S_K) change. Storage: full tensor $(N + 1)^K$, reduced via $S_K \times Z_2$ to roughly $(N + 1)^K / (2K!)$. At $K = 4$, $N = 12$: $13^4 / 48 \approx 715$ unknowns — still manageable with dense Newton.

24.2 Hellwig (CARA + supply noise)

A CARA model with Gaussian supply noise (as in Hellwig 1980) has a closed-form linear REE; MIZN's CARA mode reproduces this. Adding supply noise to the $K=3$ CRRA model is a natural extension; the price-conditioning factor in the Bayesian update gets an extra noise integral, but the 5-step architecture is unchanged.

24.3 Generalizations

- Asymmetric agents (different u_k or γ_k): breaks S_K ; revert to full tensor. Z_2 may also break depending on prior asymmetry.
- Non-Gaussian signals: signal density f_v in `signals.py` changes; rest of pipeline identical.
- Multi-asset: cube becomes higher-dimensional; sparse-grid or Smolyak tensor truncation likely needed.

Acknowledgments and References

This volume distills the research arc captured in the `mhpbreugem/fixed-point-factory` repository, branch `claude/study-fixed-point-economics-y12PB`. Specific design notes that ground the Chebyshev + symmetry + iteration treatment:

- `design_notes/chebsym/chebsym.pdf`: Chebyshev + symmetry primer.
- `design_notes/iter_methods/iter_methods.pdf`: dense Newton vs NK.
- Repo root `MIZN_DESIGN.md`, `MIZN_EXEC_LOG_DESIGN.md`: the MIZN architecture and run-logging design.
- Repo root `HANDOFF_SUMMARY.md`: complete handoff of the prior session's findings.

Key code references:

- `projects/REZN/solved_fixed_points/k3_hfree_smooth/hfree_operator.py`: the canonical $h=0$ operator (cubic spline + GL + partition-of-unity).
- `projects/REZN/solver_code/sigma_delta/v11_strict_h0_hardwired.py`: the σ - δ V11 operator from the prior session.
- Commit `78c27216`: original validation of strict- $h=0$ PR FP.
- Commit `b17b3ac1`: machine-precision nail ($\|F\| = 9.4 \times 10^{-16}$).